MPX G2

MIDI Implementation Details

Copyright 2001
All Rights Reserved

Lexicon Part Number 070-14829

Lexicon Inc.
3 Oak Park
Bedford, MA 01730
Telephone 781-280-0300
FAX 781-280-0490

TABLE OF CONTENTS

INTRODUCTION **3**
GENERAL MIDI 3
SYSEX 3
SYSEX AUTOMATION 3
MIDI **4**
PROGRAM CHANGE MESSAGES 4
General Notes *4*
Standard Program Selection *4*
MIDI MAP MODE 4
MIDI Map Extensions *5*
Program "Chains" Mode *5*
Program Number Correlation: *6*
Configurable Parameters *6*
MIDI CONTINUOUS CONTROLLERS 6
Receive *6*
Transmit *7*
MIDI NOTES 7
MIDI CLOCK 7
MIDI Clock IN *8*
MIDI Clock OUT *8*
MIDI CHANNEL 8
Receive *8*
Transmit *8*
MIDI IMPLEMENTATION CHART 9
SYSEX **10**
DATA FORMATTING NOTES 10
DEVICE INQUIRY 11
MESSAGE TABLE NOTES 12
MESSAGE TYPES 13
System Configuration Message (00) *13*
Data message (01 hex) *14*
Formatted String Message (02 hex) *16*
Object Type ID Message (03 hex) *17*
Object Description (04 hex) *18*
Object Label Message (05 hex) *20*
Requests (06 hex) *21*
MIDI Terminal Message (11 hex) *22*
Handshaking (12 hex) *22*
ALGORITHM SELECT 25
CHANGING MPX G2 PARAMETER VALUES 25
MPX G2 Parameter Examples *26*
MPX G2 - MPX R1 INTERFACE NOTES 27
MPX R1 Detection *27*
Program Selection *28*
Bypass *28*
TAP *28*
A/B *28*
Tuner *28*
Pedal *29*
Toe Switch *29*
MPX G2 - DUMPS 29
MPX G2 - Program Dump *30*
MPX G2 - All LEDs Dump *42*
MPX G2 - Display Dump *45*
MPX G2 - Custom Character Bitmap Dumps *45*
MPX G2 - Setup Dumps *46*
MPX G2 - Global Patches Dump *50*
MPX G2 - Bypass Controllers Dump *51*
MPX G2 - Remap Controllers Dump *52*
MPX G2 - Current Choices Dump *52*
MPX G2 - Patches Dump *57*
MPX G2 - Soft Row Dump *57*
MPX G2 - DISPLAY UNITS 58
Patch Destination Display Units *61*
MPX G2 CONTROL SOURCES 61
Master List of Controllers *61*
Global Patch controllers *62*
Bypass controllers *62*

Introduction

This document is the reference for the MIDI implementation used in the Lexicon MPX G2. The document is broken into two categories: General MIDI and SysEx (System Exclusive).

General MIDI

The General MIDI implementation on the MPX G2 is almost identical to the MPX 1. One exception: a "Soft Through" has been added to allow program change messages and continuous controller messages to be passed from the MIDI IN port to the MIDI OUT port, merging the data with any data generated by the MPX G2.

SysEx

The MPX G2 SysEx implementation is like the MPX 1, the MPX G2 uses the same protocol for SYSEX control and data transfer. The MPX G2 implementation is significantly more advanced/complete than the MPX 1.

The following are the most significant changes:

- All dumps in and out of the box are in the same format (the program dump on the MPX 1 is a not a standard dump).
- Front panel button messages have been broken out into separate parameters/data types to simplify glass interface creation.
- The front panel display and LEDs can be made to auto transmit their state whenever a change occurs.
- The state of all internal controllers can be made to auto transmit as a single dump at a definable interval.
- The "Meter" bridge image can be made to auto transmit as a single dump at a definable interval.
- Control Levels are treated like actual parameters and can have data sent to them with changes in the system. This is most significant for Algorithm selection.
- When a remote controller is detected, the system automatically sends out updated versions of patch Destination, Min, Mid and Max parameters when either a new algorithm is selected or a new program is selected. This keeps any remote patch editors up to date.
- Support for the MPX R1 remote has been added.
- Checksum is now included on all outgoing messages. Incoming checksum is counted but no action is taken for errors largely due to a requirement for compatibility with the MPX R1 remote.

SysEx Automation

The MPX G2 supports a special "Automation" mode that, when enabled, outputs SysEx data whenever a parameter is adjusted from the front panel. The data output is a data message that contains the current value of the parameter along with it's control address. Though the Automation feature was included in the system to "automate" controlling the same parameter on two MPX G2s at the same time, it can be quite useful for other things as well. One interesting use of the Automation feature is to identify the control address of various parameters in the system; adjust the parameter and examine the resulting SysEx message. Another potential use of the Automation feature is to trigger off-line events based on the parameter value or control address: Keeping a glass interface synchronized to the actual system, or to implement an automatic users guide.

The Automation feature can be turned on or off from the SYSTEM: MIDI menu's "Automation" parameter.

MIDI

Program Change Messages

General Notes

The MPX G2 is capable of responding to MIDI Program Change messages in several different ways depending on the selected sort criteria in the PROGRAM Mode. Other parameter settings will further affect the system's response in a particular "Sort".

Standard Program Selection

Most of the operating modes of the MPX G2 use the conventional program selection techniques (offset by -1): MIDI Program change 5 selects MPX G2 program 5, etc. In addition to the conventional MIDI Program Change message, the MPX G2 also supports program banks which extend the range of selectable programs from 0-127 to 0-16383. Since there are only 300 programs in the MPX G2, only 0-299 (1-300) are actually useable.

Program banking is achieved by using Continuous Controllers 0 and 32 to select "bank". In the MPX G2, each bank consists of 100 programs for a total of 3 banks. In the standard program selection mode, any program change message above 100 will be ignored. The MIDI bank select message (Controller #32) will be used to select the banks. Controller #0 which is normally reserved for the MSB of Bank change messages is ignored. The MPX G2 programs are assigned as follows:

Program bank 0 (1-100 Internal Presets) = 00H
Program bank 1 (101-200 Internal Presets) = 01H
Program bank 2 (201-249 Internal Presets, 250-300 User definable registers,) =02H

To select program 107 for instance, you would send Continuous Controller 32 with a value of 1 (e.g. 0xB0, 0x20, 0x01) to select bank 1 followed by Program Change 6 (e.g. 0xC0, 0x06).

Note that once the bank has been set, all further Program Change messages apply to that bank.

Also note that the MPX G2 will return to the "Sort by Number" mode if a MIDI Program Change message is received in any of the Sort modes except "MIDI Maps" and "Program Chains" (described below).

Bug Note: Bank values greater than 2 are interpreted as bank 0. They really should be ignored.

MIDI Map Mode

The MPX G2 supports 3 separate MIDI Maps. MIDI Maps allow the user to select a MPX G2 program to be loaded in response to a particular Program Change message. Each MIDI Map contains 128 locations; one for each program number supported by the MIDI Program Change message (0-127).

Example MIDI Map:

Program Change #	**MPX G2 Program #**
0	4 - Power Chords
1	7 - EnvFilter LP
2	12 - Little Wing
3	2 - Guitar Solo
...	...

The MPX G2's SYSTEM: MIDI Maps menu provides an editor that allows the assignments to be modified. Refer to the MIDI Maps section of the UI spec for additional information.

When the PROGRAM mode is set for "**Show members of MIDI Maps**", MPX G2 programs are selected based on assignments made for the particular Program Change number in the selected MIDI Map. Once MIDI Maps are selected in the PROGRAM mode, particular maps are selected by pressing the < and > buttons.

MIDI Map Extensions

In addition to simply selecting programs, the MPX G2's MIDI Maps allow MIDI Program Change messages to perform other system wide operations. Though these operations are not normally associated with Program Change, they can prove to be very useful when controlling the system from a foot controller. The following extensions are available:

Extension	**Notes**
Clear Loop	Clears the loop when the JamMan effect is in use.
Layer	Toggles the JamMan effect in and out of the Layer mode.
Replace	Toggles the JamMan effect in and out of the Replace mode.
Delay	Toggles the JamMan effect in and out of the Delay mode.
Start Stop	Stops and restarts the loop in the JamMan effect.
Tap	Performs the same operations as the front panel's TAP button.
Bypass	Toggles the system bypass on and off.
Gain Bypass	Toggles the Gain effect bypass on and off.
FX1 Bypass	Toggles the FX 1 bypass on and off.
FX2 Bypass	Toggles the FX 2 bypass on and off.
Chorus Bypass	Toggles the Chorus effect bypass on and off.
Delay Bypass	Toggles the Delay effect bypass on and off.
Reverb Bypass	Toggles the Reverb bypass on and off.
EQ Bypass	Toggles the EQ effect bypass on and off.
Insert Bypass	Toggles the INSERT bypass on and off.
Toggle A B	Performs the same operations as the front panel AB button.

Program "Chains" Mode

The MPX G2 supports 10 Program Chains. Program Chains allow users to "increment" and "decrement" through groups of up to 10 MPX G2 programs. The "incrementing" and "decrementing" is driven by assignments made for the "Pgm+" and "Pgm-" parameters which can be found in the SYSTEM:MIDI menu as an option to the "Pgm Change" parameter. Typically, these are assigned to footswitches or continuous controllers.

The MPX G2 uses MIDI Program Change messages to select a particular "Chain". Once a particular Chain has been selected, increments and decrements step through the MPX G2 programs assigned to the Chain. MIDI Program Change numbers are assigned to select Chains and MPX G2 programs are assigned to particular Chain (elements of a Chain) in the system program chains menu.

MIDI Program Change message can be used to select Chains when the PROGRAM mode is set for "**Show members of Program Chains**". Once Program Chains are selected in the PROGRAM mode, particular maps are selected by pressing the < and > buttons.

Program Number Correlation:

All incoming MIDI Program Change messages are offset by 1 relative to the programs in the MPX G2. MIDI Program Change messages begin with program 0 while the MPX G2's programs begin at 1. When a MIDI Program change value of 0 is received by the MPX G2, program 1 is loaded. Note that this can be modified by the Pgm# offset parameter which is an Option of the SYSTEM:MIDI "Pgm Change" parameter.

Configurable Parameters

The MPX G2 contains several parameters that effect how the system responds to MIDI Program Change messages.

"Pgm Change"
This parameter determines whether the MPX G2 will respond to Program Change messages or not.

"Pgm# Offset"
The Pgm# Offset (program change number offset) parameter, is a number that is added to MIDI Program Change values received by the box. If MIDI Program Change 10 (0x09) is sent to the MPX G2 and the Pgm# Offset is set for 5, program 15 would be loaded.

Note the Pgm# Offset is ignored in the MIDI Maps and Program Chains modes.

MIDI Continuous Controllers

The MPX G2 makes extended use of MIDI Continuous Controllers. In addition to dynamically changing system parameters in response to received Continuous Controller messages, the MPX G2 is even capable of generating and transmitting Continuous Controller messages.

Receive

The MPX G2 has a wide range of parameters that can be manipulated via MIDI Continuous Controller messages.

- Program Patch Sources
- Global Patch Sources
- Bypass Patch Sources
- TAP
- Program Inc/Dec
- LFO1/2 "On Source"
- A/B "On Source"

Patches

In the MPX G2, a "patch" is mapping device that allows a control *source* to manipulate a *destination* parameter. The MPX G2 provides three types of Patches: Program, Global and Bypass. Program Patches are stored and recalled with MPX G2 programs while the Global and Bypass Patches are always available. While Program and Global Patches provide continuous control over their target *destination* parameters, Bypass Patches simply allow the effect and system bypasses to be toggled on and off using Continuous Controllers.

In addition, several internal control sources (LFOs, Envelopes, etc.), MIDI Continuous Controllers can be selected as *sources* for Global, Program and Bypass Patches. All Continuous Controllers from 1-31, 33-119 can be used as Patch sources. Controllers 0 and 32 are excluded because they control program banking and Controllers over 120 are reserved.

TAP

The MPX G2 allows a MIDI Continuous Controller to actuate the system's TAP function. The selection of Controller to use is made from the EDIT: Tempo menu "Tap Source" parameter. A threshold can be set from the parameter's option "Source Level" parameter. A "Tap" event is detected each time the controller goes from below the Source Level to above it.

Program Inc/Dec

The MPX G2 allows a MIDI Continuous Controller to increment and/or decrement programs. The selection of Controller to use is made from the SYSTEM: MIDI "Pgm Change" parameter's option menu. The right hand page of the option menu contains the "Pgm+" and "Pgm-" parameters to which the Controllers are assigned. Programs are incremented or decrements when the Controller crosses from less than 64 to 64.

LFO 1 / LFO 2 "On Source"

The MPX G2 allows a MIDI Continuous Controller to control whether an LFO is on or off. The selection of Controller to use is made from the EDIT: Controllers LFO1/2 menu. The last parameter on the row is the "OnSrc" parameter. When the Controller is below the value of the "OnLvl" parameter, the LFO is turned off. When the Controller hits the value of the "OnLvl" parameter or greater, the LFO turns on.

AB "On Source"

The MPX G2 allows a MIDI Continuous Controller to trigger an A-B ramp. The selection of Controller to use is made from the EDIT: Controllers A/B menu. The last parameter on the row is the "OnSrc" parameter. When the Controller crosses from below the value of the "OnLvl" parameter to above it, the A/B ramp begins.

Transmit

The MPX G2 is capable of actually generating MIDI Continuous Controller messages. For all of the Patch sources the system supports, Continuous Controllers can be transmitted. The SYSTEN: MIDI menu contains a pair of parameters grouped under Ctl Send which allow you to select the Continuous Controller number that the Patch source controllers will transmit as. The one limitation is that only the Patch Sources for Program Patches are transmitted and only when the program they are used in, is loaded.

MIDI Notes

The MPX G2 can use incoming MIDI note ON events as control sources for program patches. The note number is used to drive the patch (just as a MIDI controller "value" drives a patch). incoming MIDI notes must be on the MIDI channel defined by the SYSTEM: MIDI "Receive" parameter. The following variations of MIDI notes are available as patch sources:

Name	**Description**
Last (note)	This is the last MIDI note that was received by the system.
Low (note)	This is the lowest note currently being played (in a chord)
High (note)	This is the highest note currently being played (in a chord)

MIDI Clock

The MPX G2 is capable of both generating and synchronizing to MIDI clock.

MIDI Clock IN

The MPX G2 can use incoming MIDI clock to set the system's tempo, which in turn can control parameters throughout the system. To use external MIDI clock with the MPX G2, the EDIT: Tempo "Source" parameter must be set to "MIDI". With the Source set for MIDI, the Tempo "Rate" (in BPM) is calculated from the incoming MIDI clock. The settings of these parameters are stored and recalled with each program unless the SYSTEM: Mode Tempo parameter is set for "Global".

The Tempo "Rate" can be used by several parameters throughout the system to provide unified synchronization. Examples include the Delay "Time" parameter which can be based on Tempo Rate and virtually all "Rate" parameters used in LFOs, Chorus, etc. These parameters can be put into the "ratio" mode (which uses the Tempo Rate) by pressing the Options button when the parameter is selected.

Incoming MIDI clock can also be used to define the loop size when using the JamMan algorithm in the Delay effects block. In this case, the loop size will be determined by the tempo of the incoming MIDI clock and the current value of the Edit:Tempo "Beat Value" parameter. This parameter defines the musical size of an MPX G2 "beat" (two successive presses of the TAP button). If a two-measure loop in 4/4 were being used, the Beat Value parameter would be set for "8 Beats". With the Tempo Source parameter set for "MIDI", only a single press of the TAP button would be required. The loop would automatically be closed by the system when the equivalent of 8 musical beats, in MIDI clocks, was received.

MIDI Clock OUT

The MPX G2 can actually generate a stable MIDI clock that can be used by other equipment to synchronize itself to the G2. The output of MIDI clock can be turned on and off from the SYSTEM: MIDI menu via the "Clock Send" parameter. The tempo of the outgoing clock is based on the current value of the Tempo Rate parameter. Refer to the MIDI Clock IN section for additional information on how tempo is used by the system.

When the JamMan effect is loaded in the Delay block, MIDI clock (and program Tempo Rate) is defined by the size of the loop. In addition, a MIDI Start command is sent when a loop is tapped in to start any connected drum machines or sequencers. A MIDI stop command is sent when the loop is reset/cleared. When using the outgoing MIDI clock with the JamMan effect, the EDIT: Tempo "Beat Value" parameter must be set for the number of beats which appear in the loop in order for the connected equipment to stay in sync with the loop.

MIDI Channel

The MPX G2 allows a separate MIDI channel to be defined for incoming (Receive) data and outgoing (Transmit) data. The Receive and Transmit MIDI channels are set from the G2's SYSTEM: MIDI menu.

Receive

The "Receive" channel defines the MIDI channel to which the system will respond for program changes, continuous controllers and MIDI notes. In addition to being selected manually, the Receive MIDI channel can also be set automatically from incoming data from the Patch Source's OPTION menu. This menu allows both the MIDI channel and a controller number to be "learned" for use as a source to a program Patch. The learn feature is also available for the Global Patch's Source parameter. Legal values of 1-16, Off and Omni are available.

Transmit

The "Transmit" channel defines the MIDI channel to which the system will generate general MIDI messages. At present, the only general MIDI message that the system is capable of generating is continuous controller messages. The SYSTEM: MIDI menu contains a utility ("Ctl Send") that allows continuous controller numbers to be assigned to each of the control "sources" that are available as patch control sources. When these control sources are used in a program patch, it is automatically transmitted as the controller assigned in this utility.

MIDI Implementation Chart

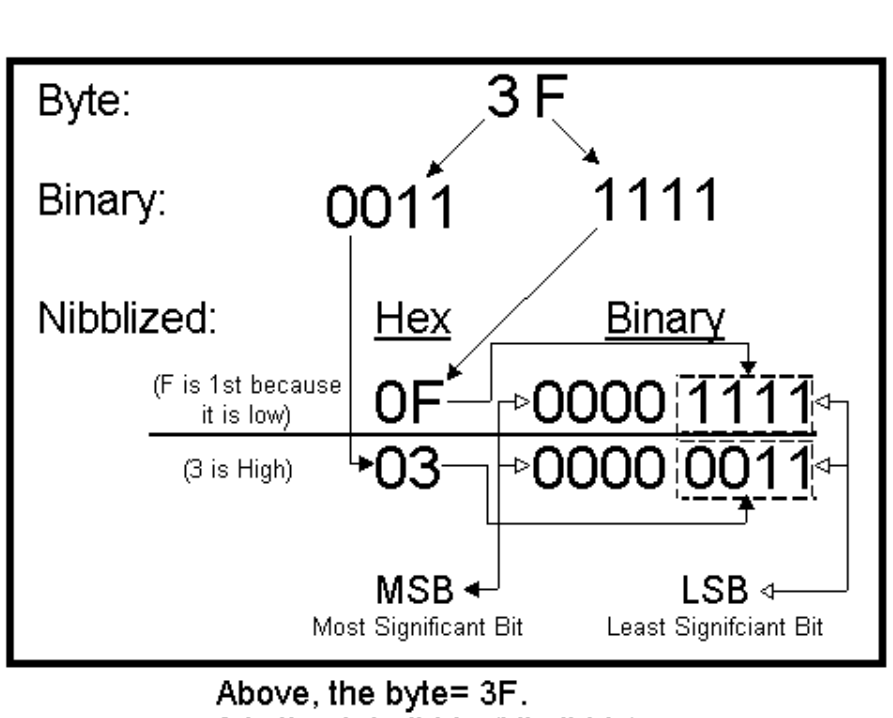
Function	**Transmitted**	**Recognized**	**Remarks**
Basic Channel Default Changed	1 1-16	1 1-16	
Mode Default Messages Altered	Mode 1 X	Mode 1, 3 X X	
Note Number	0-127	0-127	Last Note, Low Note, High Note are used as controllers
Velocity Note ON Note OFF	O X (Off-9n v=0)	O	used as controller
After Touch Keys Channel	X X	X O	used as controller
Pitch Bend	X	O	used as controller
Control Change	OX	OX	1-119 for MIDI transmit
Program Change Bank Select	O O	0-99 O	100-127 ignored Program message 0-99 = Program change 1-100. Program maps selectable via MIDI bank select messages
System Exclusive :Lexicon :real-time :NonRealTime	OX X Device ID	OX X Device Inquiry	Product ID = 0F Device ID
System Common :Song Pos :Song Sel :Tune	X X X	X X X	
System Real Time :Clock :Commands	OX X	OX OX*	*START, STOP and CONTINUE are patchable as a switch: START/CONTINUE = 127, STOP = 0
Aux Messages :Local ON/OFF :All Notes OFF :Active Sense :Reset All Controllers	X X X OX	X O X OX	
Notes: Mode 1: OMNI ON/POLY Mode 2: OMNI ON/MONO Mode 3: OMNI OFF/POLY Mode 4: OMNI OFF/MONO O: Yes OX: selectable X: NO			

SYSEX

Data Formatting Notes

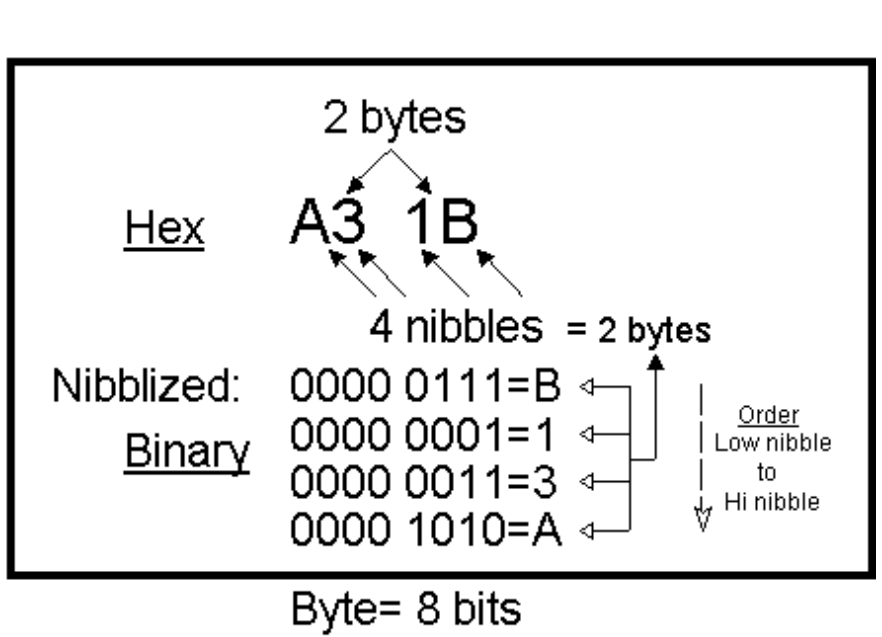
In an effort to simplify handling and viewing messages, with the exception of each message's header (1st 5 bytes) and "end of SYSEX" (the last byte in the message) message, all data appears in a "nibblized" format. Ultimately, this makes interpreting the data extremely straightforward: un-nibblize then look for structured fields.

By nibblized" we mean that each byte of data is split into two separate bytes, each containing one nibble of the original byte. The high nibble of the resulting bytes are filled with zeros as follows:



Above, the byte= 3F.
3 is the 1st nibble (Hi nibble)
F is the 2nd nibble (Low nibble)

For data organized as 16 bit "words" (2 bytes), are sent low nibble of the low byte followed by high nibble of the low byte, followed by the low nibble of the high byte and finally the hi nibble of the high byte. For example:



Byte= 8 bits
Nibble= 4 bits

The implication of this is that some messages will be unnecessarily long. We recognize this but felt the trade off was worth it considering the complexity of the protocol, the box itself and the actual data contained in the messages (dumps, etc.).

Device Inquiry

The product supports Non-Real Time System Exclusive General Information "Device Inquiry" defined in the "MIDI 1.0 Detailed Specification 4.2". When the product receives the following message:

0xF0	Begin SysEx message
0x7E	Universal System Exclusive Non-real time header
0x0n	MIDI Channel (use 7F for any MIDI channel)
0x06	General Information (sub-ID#1)
0x02	Device Inquiry message (sub-ID#2)
0xF7	End of SysEx

The product will respond with the following message:

0xF0	Begin SysEx message
0x7E	<channel> Universal System Exclusive Non-real time header
0x0n	MIDI Channel
0x06	General Information (sub-ID#1)
0x02	Identity Reply (sub-ID#1)
0x06	Lexicon's (Manufacturer's) System Exclusive id code
0x00 0x00	Device Family Code
0bbb bbbb 0x00	Device Family Member Code
0x0n	Major Software Version number
0x0n	Minor Software Version number
0x0n	Software Development Phase*
0x00	unused
0xF7	End of SysEx

The Software Development Phase is defined as follows:

0 - Released Software
1 - Pre-Alpha Software
2 - Alpha Software
3 - Beta Software
4 - Gamma Software

All software released to the general public is 0, "Released".

Message Table Notes

Throughout this document, SysEx messages are broken out using "Message Tables". These tables provide a consistent method of describing the contents of each message class. The tables are organized with separate "rows" for each related group of bytes (referred to as "fields") and "columns" which break up the attributes of each field. For example, the "System Configuration" message table is defined as follows:

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0F	Product ID	
4	0bbb bbbb	Device ID	0-127
5	00	System Configuration	
6-7	0n	Major Version (1 byte)	The number that appears to the left of the decimal point on the power up display. Example: V1.00
8-9	0n	Minor Version (1 byte)	...
...	...	...	...

The first column indicates the "**Byte #**" of a particular field. Whenever possible, this column provides an index into the packet for that field (row). In some cases, this index is not fixed due to fields, which have a variable size, which proceed it.

The second column indicates the "**Value**" of the particular field, usually in hex (sometimes in binary). In some cases the value is fixed number ("Lexicon ID") while others can change from message to message. "Values" that can change are represented using a combination of numbers and letters. Binary numbers, for instance, are represented using the lower case letter 'b' for each variable bit in the value ("Device ID: 0bbb bbbb. The 'b's can be '1's or '0's). Hex numbers, which can use different numbers in the lower nibble, are represented with lower case letter 'n's (Major Version: 0n. The 'n' can be any hex number between 0 and F).

Although only a single "0n" is indicated in the "Value" column for certain fields, these fields will often consist of multiple bytes (each of which uses the format: 0n). This is due, in part, to the nibblizing scheme used in this SysEx implementation (see the previous section), but also due to capability of a single field to describe a group of more than 1 byte (strings, for example). In these cases, the "Description" column usually indicates how many bytes of actual data are contained in the field. I say "actual" data because all of the data bytes following the header (bytes 1-5) are in the "nibblized" format: each byte is broken up into 2 separate bytes. The first byte contains the low nibble and the second byte contains the high nibble. The "number of bytes" indicated in this column, therefor, is for the data *after* it has been de-nibblized.

Message Types

The System Exclusive for the MPX G2 is implemented through the use of standard message "types" which are common for current Lexicon products. In each SysEx message, the "type" identifier appears just after the standard SysEx header (Start of SysEx: F0, Company ID, Product ID, Device ID) and defines how the remaining data should be interpreted. MPX G2 uses the following standard message types:

System Configuration Message (00)

The System Configuration Message provides a method for extracting information about the software in the system from the box via SYSEX. As such, this message can only be requested from the system and will be ignored if sent TO the system.

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	00	Message Type	System Configuration
6-7	0n	Major Version (1 byte)	The number that appears to the left of the decimal point on the power up display. Example: V1.00
8-9	0n	Minor Version (1 byte)	The number that appears to the right of the decimal point on power up display. Example: V1.00
10-25	0n	8 Character Time string (8 bytes)	Time of the code build in ASCII format: xx:yy:zz xx = Hour yy = Minute zz = Second Example: 17:51:03
26-47	0n	11 Character Date string (11 bytes)	Date of the code build in ASCII format: xxx:yy:zzzz xxx = Month yy = Day zzzz = Year Example: May 10 1996
48-51	0n	Number of Object Types (2 bytes)	The total number of "Object Types" in the product. (if this field is 0, look elsewhere for the count.)
52-55	0n	reserved	
56-59	0n	Number of Control Levels used (2 bytes)	This is the maximum number of control levels used by the system
60-63	0n	Number of additional bytes appear in this message (after this byte)	(all the following bytes except F7 are optional)
64-65	0n	Protocol version	(if this byte is not present, version 1 is implied)
66-73	0n	commands supported	This is a bitmapped field indication which commands are supported (1) and which are not (0): Bit Command 0 System Configuration 1 Data Message 2 Data Display Message 3 Data Type Message 4 Data Description Message 5 Data Label 6 Requests 7 Rebuild Control Tree 8 Dump Data Types 9 Dump Control Addresses 10 Utility Message 11 Memory Tool 12 MIDI Terminal 13 Handshake
74-75	0n	Maximum number of "Units"	This indicates the maximum number of "Units" that are used on any "Data Type" in the system. (MPX G2 has 5)
76-77	0n	Maximum dump size	Optional 16 bit field indicating the maximum dump size used by the system.
78	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the Message Type.
--	F7	End of Sysex	The byte number is one less if the checksum field is not used.

Number of Object Types

The "Number of Object Types" tells you how many types of data can be moved in and out of the product. An editor program may use this to request object description (04 hex) of all the Object Types in the system.

The remaining information in the message can be used by the controlling program/system to detect software updates as well as inform the user of the exact ROM version being used.

Request

There are no arguments to the System Configuration "request". The "request" message is as follows,assuming the Device ID is 0 and the product is the MPX G2: 0x0F:

Request
F0 06 0F 00 06 00 00 00 00 00 00 00 00 F7

Data message (01 hex)

This message allows all types of data to be passed to and from the system. Typically this message type is used to change parameter values in a system remotely (automation, etc.) and to dump data (programs, setups, etc.) in and out of the box.

Note - To help simplify the following descriptions, the actual data defined by the Object is referred to as the "primary" data. If the Object uses an option, the option's data immediately follows the "primary" data.

Like most of the messages, this message uses control addresses to specify exactly which Object (parameter, etc.) type in the system we are trying to access.

Transmit + Receive

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127 transmit, 0-126 receive
5	01	Message type	Data Message
6-9	0n	Number of bytes (2 bytes)	This is the number of bytes in the Object and its option (if available). This is a 16 bit field so up to 64K bytes is supported.
10-x	0n	data	This is the actual data and (if available) it's option's data. Option data always appears after the primary data. Use the number of bytes field of the Object and option's description to determine where the primary data ends and the option begins.
--	0n	Number of control levels (2 bytes)	This field defines how many control levels are used in the Object's address. This is a 16 bit field so the address can have up to 64k digits (control levels) (max is defined in the "System Configuration" message)
--	0n	Control level 0 (A) (address) (2 bytes)	This is the first control level. It defines the level 0 (A) control address of the Object. The control level addresses are 16 bit fields so the control addresses can be 64k deep. Note that letters are used to help differentiate between levels and their values.
--	0n	Control level 1 (B) (address) (2 bytes)	Same as previous
--	0n	Control level 2 (C) (address) (2 bytes)	Same as previous
--	0n	--- up to Control level 65535	Same as previous
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the "Message Type".
	F7	End of Sysex	

Note that the "data" includes any "Option" data associated with the Object. For example a 16 bit "Data Type" with an 8 bit option would contain 3 bytes of data. The Object Type of both the main data object and its option must be used to correctly interpret the "data" in this message.

The "Control Levels" are used in the request message for this packet:

0n - **Number of Control Levels** (lo nib) (Nibble 1 of argument)
0n - Number of Control Levels (lo mid nib) (Nibble 2 of argument)
0n - Number of Control Levels (hi mid nib) (Nibble 3 of argument)
0n - Number of Control Levels (hi nib) (Nibble 4 of argument)
0n - **Control Level 0 (A)** (lo nib) (Nibble 5 of argument)
0n - Control Level A (lo mid nib) (Nibble 6 of argument)
0n - Control Level A (hi mid nib) (Nibble 7 of argument)
0n - Control Level A (hi nib) (Nibble 8 of argument)
0n - **Control Level 1 (B)** (lo nib) (Nibble 9 of argument)
0n - Control Level B (lo mid nib) (Nibble 10 of argument)
0n - Control Level B (hi mid nib) (Nibble 11 of argument)
0n - Control Level B (hi nib) (Nibble 12 of argument)

Formatted String Message (02 hex)

This message provides a formatted string version of an Object's (parameter, etc.) value. This message was included to allow external systems to display parameter values without having to worry about how to convert the raw data to a formatted string. This is particularly useful for parameters who's values only appear as lists of text strings.

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	02	Object Display	Message type
6-9	0n	Number of characters (2 bytes)	This is the number of chars (bytes) in the string
10-x	0n	String characters	"Number of characters" of nibblized ASCII characters
--	0n	Number of control levels (2 bytes)	This field defines how many control levels are used in the Object's address. This is a 16 bit field so the address can have up to 64k digits (control levels)
--	0n	Control level 0 (A) (address) (2 bytes)	This is the first control level. It defines the level 0 (A) control address of the Object. The control level addresses are 16 bit fields so the control addresses can be 64k deep. Note that letters are used to help differentiate between levels and their values.
--	0n	Control level 1 (B) (address) (2 bytes)	Same as previous
--	0n	Control level 2 (C) (address) (2 bytes)	Same as previous
--	0n	--- up to Control level 65535	Same as previous
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the Message Type.
	F7	End of Sysex	

The alternative to using this message is to use the Data Message and to interpret the data off-line.

The "Control Levels" are used in the request message for this packet:

0n - **Number of Control Levels** (lo nib) (Nibble 1 of argument)
0n - Number of Control Levels (lo mid nib) (Nibble 2 of argument)
0n - Number of Control Levels (hi mid nib) (Nibble 3 of argument)
0n - Number of Control Levels (hi nib) (Nibble 4 of argument)
0n - **Control Level 0 (A)** (lo nib) (Nibble 5 of argument)
0n - Control Level A (lo mid nib) (Nibble 6 of argument)
0n - Control Level A (hi mid nib) (Nibble 7 of argument)
0n - Control Level A (hi nib) (Nibble 8 of argument)
0n - **Control Level 1 (B)** (lo nib) (Nibble 9 of argument)
0n - Control Level B (lo mid nib) (Nibble 10 of argument)
0n - Control Level B (hi mid nib) (Nibble 11 of argument)
0n - Control Level B (hi nib) (Nibble 12 of argument)

Object Type ID Message (03 hex)

This message is transmitted from the system in response to a request for the Object Type ID at a specific Control Address. This allows external equipment to build a control tree of the system: request the Object Type ID (this message) at an address, check it's description to see if it is a control "branch" or a "leaf". If the address contains a "branch" flag, you know you can go one level deeper until hitting a "leaf".

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product	
4	0bbb bbbb	Device ID	0-127
5	03	Message type	Object Type ID
6-9	0n	Object Type ID(2 bytes)	The max value here will be the one less than the "Number of Object Types IDs" field of the "System Configuration" message
--	0n	Number of control levels (2 bytes)	(optional)This field defines how many control levels are used in the address. This is a 16 bit field so the address can have up to 64k digits (control levels)
--	0n	Control level 0 (A) (address) (2 bytes)	(optional)This is the first control level. It defines the level 0 (A) control address of the Object. The control level addresses are 16 bit fields so the control addresses can be 64k deep. Note that letters are used to help differentiate between levels and their values.
--	0n	Control level 1 (B) (address) (2 bytes)	same as previous
--	0n	Control level 2 (C) (address) (2 bytes)	same as previous
--	0n	--- up to Control level 65535	same as previous
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the Message Type.
---	F7	End of SysEx	

The "Control Levels" are used in the request message for this packet:

0n - **Number of Control Levels** (lo nib) (Nibble 1 of argument)
0n - Number of Control Levels (lo mid nib) (Nibble 2 of argument)
0n - Number of Control Levels (hi mid nib) (Nibble 3 of argument)
0n - Number of Control Levels (hi nib) (Nibble 4 of argument)
0n - **Control Level A** (lo nib) (Nibble 5 of argument)
0n - Control Level A (lo mid nib) (Nibble 6 of argument)
0n - Control Level A (hi mid nib) (Nibble 7 of argument)
0n - Control Level A (hi nib) (Nibble 8 of argument)
0n - **Control Level B** (lo nib) (Nibble 9 of argument)
0n - Control Level B (lo mid nib) (Nibble 10 of argument)
0n - Control Level B (hi mid nib) (Nibble 11 of argument)
0n - Control Level B (hi nib) (Nibble 12 of argument)

Object Description (04 hex)

This message allows external equipment to obtain information about each of the Object Types in the system. Each Object Type used by the system is assigned a unique number that can be used to identify it much like menus and dialog boxes are assigned ids in Windows and Macintosh software. These don't represent the Objects themselves but rather the attributes of an Object. In this way an Object Type can be reused for several parameters or other data objects in the system. Typically, an outside developer, after requesting the "System Configuration" message will request this message once for each legal Object Type in the system building a database of all Object Types used by the system. Using the Object Description, the control tree for the system can be built.

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	04	Message Type	Object description
6-9	0n	Object Type (2 bytes)	max 65536 types
10-11	0n	Number of Characters in Name (1 byte)	max 255 characters
12 - n	0n	Name of Object (number of characters nibbled)	ASCII (note that some LCDs use 0-7 for custom characters. 0, therefor, should not be interpreted as the end of the string)
	0n	Number of bytes (2 bytes)	Size of the data associated with the Object (max 65535 bytes)
	0n	Control Flags (1 byte)	see below
	0n	Option Object Type (2 bytes)	Uses standard Object Type IDs (numbers). Note that a value of 0xFFFF indicates no options are used.
	0n	Number of units/limits (1 byte)	max 255 units/limits
	0n	1st Minimum Value (2 bytes)	
	0n	1st Maximum Value (2 bytes)	
	0n	1st Display units (2 bytes)	see below
	0n	2nd Minimum Value (2 bytes)	
	0n	2nd Maximum Value (2 bytes)	
	0n	2nd Display units (2 bytes)	
	0n	nth Minimum Value (2 bytes)	
	0n	nth Maximum Value (2 bytes)	
	0n	nth Display units (2 bytes)	
	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the Message Type.
	F7	End of Sysex	

Object Type Number (Bytes 6-x)

This is the "id" of this Object Type. Each actual Object Type in the system is assigned a unique Object Type ID to which is used to reference its description structure.

Number of Object Name Characters (Bytes 10-11)

This is the number of characters in the name but also tells you where the Object "Name" field ends and where "Number of bytes" begins. Not all Object names are of the same size; audio parameter Objects have 5 character names while most others have 11 characters.

Object Type Name (Bytes 12-x)

This is actual text string used to describe the Object Type. Note that this string is NOT null terminated but should be padded with spaces (ASCII 20 hex). In most cases, these are the strings that appear on the system's front panel for a given Object Type. This would account for the sometime terse and often duplicate names for Objects. Usually Objects with the same names have other attributes that are different.

Number of Bytes

This is the size of the data associated with the Object in bytes. Most Objects used, as parameters are 1 or 2 bytes. Objects that are bigger include the "Program" name, the "Setup" name and other "dump" Data Types. Though not reflected in this number, the data bytes for an Object's "Option" s are included in data messages for Objects with options. When dealing with messages, the description for the option's Object Type must be referenced to correctly interpret the data (number of bytes in the option Data Type, min/max value, etc.).

Note that some "option" Objects ("Rate units", "DRate units", etc.) actually have 0 bytes. In these Objects, the "data" is actually contained in the MSB of the root Object's data.

When Objects are used for parameters, the MSB of the Display Units field indicates if the parameter is signed or unsigned. See note below.

Control flags

The control flags are used to define several attributes that are either on or off (0 = NO, 1 = YES). The following bits have been defined:

Bit	**Mask**	**Name**	**Description**
0	0x01	Patchable	This object can be controlled by a patch
1	0x02	Automation	This Object is transmitted when automation is turned on
2	0x04	Control level	This Object is a control level
3	0x08	Bottom Control Level	This is the last control level on a branch before an editable Object
4	0x10	Uses Tempo	This Object is affected by tempo changes
5	0x20	Wrapping	The control for this Object can wrap around from it's minimum value to it's max and vise versa)
6	0x40	Soft Row Assignable	The control for this Object can be placed in a "Soft Row" (simplified user interface)

Options Object Type

When an Object has an option, an Object Type defines the attributes of the option itself. This field of the description contains the id for the Object Type that describes the option parameter. Often, the value of the option parameter must be evaluated *before* the parameter's value can be correctly interpreted ("Delay" units for instance). A value of 0xFFFF indicates that no option Object is used.

Number of Units/Limits

Some Objects in a system can operate in different "unit" types. An example of this is delay time, which can be set in milliseconds, a ratio of beats per measure relative to the current tempo, feet, meters and tapped in milliseconds. The system only has a fixed amount of actual delay memory and the steps of each of these represents different amounts of time so the limits for each is different. This field defines how many unit types are available for this Object. This value will also tell you how many mins, maxs, and display units are included in this packet.

If "Number of Units/Limits" is more than 1, the "Option" is used to determine which unit type is used and what they are called. Use the "String" version of the "Option" to get a text description of the current "Unit".

Minimum Value

This is the minimum allowable value for this Object type. Note that the values are expressed as signed words that must be *cast* to signed or unsigned bytes if the Object is a single byte parameter. Also note that the min and max values are only meaningful for 1 and 2 byte parameters. Typically, the min and max fields are defaulted to 0x0000 and 0xFFFF respectively for "dumps".

Maximum Value

This is the maximum allowable value for this Object type.

Display Units Type

This field specifies a specific display "type" for this Object. Like the Objects, the display "types" define a display technique that can be used for more than one Object (parameter, etc.).

The Display Units Type field also indicates if the parameter is **signed or unsigned**. If the MSB of the Display Units Type field is 1 (8000 hex), the parameter value is signed otherwise, the parameter value is unsigned.

Object Label Message (05 hex)

This message allows you to get a string describing the Object at a given control level. This message was included to simplify navigation of the control tree when the controlling system has limited memory available. The string returned is the same as the "name" field of the Object description message.

Transmit only

Byte #	**Value HEX**	**Description**	**Notes**
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	05	Object Label	Message type
6-9	0n	Number of characters	This is the number of chars (bytes) in the string
10-x	0n	Object Label String	This is the actual string (nibblized) so it is the "Number of characters" times 2 long
--	0n	Number of control levels (2 bytes)	This field defines how many control levels are used in the Object's address. This is a 16 bit field so the address can have up to 64k digits (control levels)
--	0n	Control level 0 (A) (address) (2 bytes)	This is the first control level. It defines the level 0 (A) control address of the Object. The control level addresses are 16 bit fields so the control addresses can be 64k deep. Note that letters are used to help differentiate between levels and their values.
--	0n	Control level 1 (B) (address) (2 bytes)	same as previous
--	0n	Control level 2 (C) (address) (2 bytes)	same as previous
--	0n	--- up to Control level 65535	same as previous
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the "Message Type".
	F7	End of Sysex	

The "Control Levels" are used in the request message for this packet:

The "Control Levels" are used in the request message for this packet.

0n - **Number of Control Levels** (lo nib) (Nibble 1 of argument)
0n - Number of Control Levels (lo mid nib) (Nibble 2 of argument)
0n - Number of Control Levels (hi mid nib) (Nibble 3 of argument)
0n - Number of Control Levels (hi nib) (Nibble 4 of argument)
0n - **Control Level 0 (A)** (lo nib) (Nibble 5 of argument)
0n - Control Level A (lo mid nib) (Nibble 6 of argument)
0n - Control Level A (hi mid nib) (Nibble 7 of argument)
0n - Control Level A (hi nib) (Nibble 8 of argument)
0n - **Control Level 1 (B)** (lo nib) (Nibble 9 of argument)
0n - Control Level B (lo mid nib) (Nibble 10 of argument)
0n - Control Level B (hi mid nib) (Nibble 11 of argument)
0n - Control Level B (hi nib) (Nibble 12 of argument)

Requests (06 hex)

The request message class was included to allow external systems "request" information/data from the System. The System responds to "requests" by outputting a SYSEX message with the same message type as the "Request type" which identifies the request.

Receive Only

Byte #	Value HEX	Description	Notes
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	06	Message Type	Requests
6-7	0n	Request type (1 byte)	
8-9	0n	Argument 1 (1 byte)	Message requests require arguments
10-11	0n	Argument 2 (1 byte)	
---	0n	Argument n (1 byte)	no theoretical limit to number of arguments
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the "Message Type".
---	F7	End of Sysex	

Note that the "Request Type" of the request is the "Message Type" of the response. The arguments vary depending on the "Request Type". As stated above, the "Request Type" is also the "Message Type" of the SYSEX message that the System will transmit in response to the request. Refer to the specific "Message Type" sections for information about the arguments to those requests.

MIDI Terminal Message (11 hex)

This message allows text messages (reports) to be transmitted from the system and sent to the system. In general, outgoing messages are used to report information during software development. If the system software detects an error condition, for instance, this message could be used to report the error. In addition to this, strings can be sent *to* the system, which it will optionally respond to. Transmit and receive capabilities described here will, in effect, provide a command line interface (CLI) to the system (or "MIDI Terminal" interface). The "in-house" version of the MIDI spec for a system which responds to incoming string "commands", includes a full description of these commands. Note that the transmission and reception of strings can be turned on and off using the "Handshaking" message.

Transmit + Receive

Byte #	Value HEX	Description	Notes
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	11	Message Class	MIDI Terminal Message
6-7	0n	Number of Characters (1 byte)	7-263
8-x	0n	Characters	"Number of Characters" of ASCII characters
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all preceding bytes in the message following the "Message Class".
	F7	End of Sysex	

This message cannot be requested of the system.

Handshaking (12 hex)

This message provides a mechanism for synchronizing certain aspects of one system to another. The most significant use of this is during bulk dumps to a system where the "sending" unit (an external editor, another System, etc.) could conceivably send the data too fast for the receiving system to process it. In these cases the receiving system transmits a BUSY handshake message while it internally processes the last dump. When it is ready to receive more data it transmits a READY handshake message. Using this simple handshaking, data can be transmitted at the fastest possible rate with no chance of corrupting the data (buffer overflows, etc.).

Transmit + Receive

Byte #	Value HEX	Description	Notes
1	F0	Sysex ID (Start)	
2	06	Lexicon ID	
3	0bbb bbbb	Product ID	
4	0bbb bbbb	Device ID	0-127
5	12	Message Class	Handshake Message Class
6-7	0n	Handshake Command (1 byte)	
---	0n	Checksum	This is an optional "Checksum" field. The checksum is the low 7 bits of the sum of all proceeding bytes in the message following the "Message Class".
	F7	End of Sysex	

The following handshake commands have been defined:

0	No Operation (just a dummy)
1	Are you There
2	I'm Alive
3	I'm Busy. Please wait
4	I'm ready (used after a wait)
5	Error. Re-send data.
6	Enable Small (8 bit) address mode
7	Enable Large (16 bit) address mode (default)
8	Transmit control tree
9	Transmit linked parameters
10	Don't transmit linked parameters
11	Turn ON all MIDI output
12	Turn OFF all MIDI output
13	Turn ON MIDI Terminal
14	Turn OFF MIDI Terminal
15	Turn ON "Auto Display"
16	Turn OFF "Auto Display"
17	Turn ON "Flash ROM Write Mode" (unlock step 1)
18	Turn ON "Flash ROM Write Mode" (unlock step 2)
19	Turn ON "Flash ROM Write Mode" (unlock step 3)
20	Turn OFF "Flash ROM Write Mode"

0 - NOP
This does nothing but may be used to notify a controlling system that a new system has been connected.

1 - Are You There?
This is sent by a controlling system to a product to determine if it is (still) connected. An "IM_ALIVE" is expected back.

2 - I'm Alive
This is transmitted by a system to notify the outside world that it exists. It can be transmitted automatically or in response to "ARE_YOU_THERE".

3 - I'm Busy. Please Wait
This is sent by a system if it is not ready to receive any more data. Typically this is used during dumps where the system is storing the data.

4 - Ready
This is sent by a system if it is "ready" to resume a dump for example.

5 - Error
This is returned by a system if bad data was detected. Sending system should re-send the data.

6 - Enable Small (8 bit) address mode
7 - Enable Large (16 bit) address mode
These put the system into the small and large address modes. In the default "Large" mode, the message fields for number of control levels and the actual control address are 16 bit values (0 - 65535). In the "Small" mode, these fields are reduced to 8 bits (0 - 255) making many of the common messages significantly smaller.

8 - Transmit the control tree
When a system receives this message it will transmit the ""Data Type" Type" message (including the normally optional control level information) for each node in the system. The system must watch for "BUSY" messages and wait for the subsequent "READY" message if one arrives.

9 - Transmit Linked Parameters
This is sent by the controlling system to the product to place the system into a "Transmit Linked "Data Types" mode. When this mode is turned on, the system will automatically transmit ""Data Type" Data" messages for all "linked" "Data Types. "Linked" "Data Types make individual changes but also force other "Data Types to change their values. When the system receives a ""Data Type" Data" message for a "Data Type" which is "linked" to others, ""Data Type" Data" messages for the "other" "Data Types are transmitted back to the controller so that it will have current values.

10 - Don't Transmit Linked Parameters
This shuts off the "Transmit Linked "Data Types" mode.

11 - Turn ON All MIDI Output
This instructs the connected system to enable the output of all MIDI data. Internal modes and settings are updated accordingly.

12 - Turn OFF All MIDI Output
This instructs the connected system to disable the output of all MIDI data. Internal modes and settings are updated accordingly.

13 - Turn ON the MIDI Terminal Mode
14 - Turn OFF the MIDI Terminal Mode
These turn on and off the "MIDI Terminal" mode (see MIDI Terminal Message (11 hex)).

15 - Turn ON the "Auto Display Mode"
When a system receives this message, a global mode is set which generates a display dump message for the product whenever the display (LEDs, LCD, etc.) change.

16 - Turn OFF the "Auto Display Mode"
This turns off the auto display mode in the system.

17 - Turn ON the "Flash ROM Write Mode 1" (unlock step 1)
18 - Turn ON the "Flash ROM Write Mode 2" (unlock step 2)
19 - Turn ON the "Flash ROM Write Mode 3" (unlock step 3)
These turn on the flash ROM write mode in the system. This shuts down normal operation of the system and prepares it for a Flash Update procedure.

20 - Turn OFF the "Flash ROM Write Mode"
This turns off the flash ROM write mode then reboots the connected system.

21 - Run Flash Command
This runs the current flash command specified by the Command Field of the Flash ROM code header.

22 - Clear Checksum
This clears the running checksum used verify valid Flash ROM data.

Example:
This message can also be used to determine if a system is connected by sending the HANDSHAKE_ARE_YOU_THERE message to the system as follows:

F0 06 0F 00 **12 01** F7

This more specialized use of the handshake is one of the reasons the standard MIDI handshake (used for Sample dump, etc.) is not used.

Algorithm Select

Changing algorithms on the MPX G2 is handled by treating one of the tree nodes as a parameter, sending it data. To select the "Detune (M)" algorithm for FX 1 for instance, you would send the following message:

F0 06 09 00 01 01 00 00 00 01 00 02 00 00 00 00 00 00 00 00 00 00 00 00 F7

This breaks out as follows:

Raw data (hex)	Description
F0 06 09 00 01	Basic header: company, product and device id followed by the message class
01 00 00 00	the number of bytes in the message (nibblized)
01 00	the actual data: algorithm 1 (Detune (M))(nibblized)
02 00 00 00	the number of control levels in the address (2)(nibblized)
00 00 00 00	the first control level (program: 0 or system: 1)(nibblized)
00 00 00 00	the second control level (effect type)(nibblized)
F7	end SysEx

The first control level is always 0 for parameters associated with the running program and 1 to access system level parameters on the MPX G2.

When the first control level is 0 (program), the second level represents the effect type for values up to 6.

The data portion of the message represents the algorithm we are selecting. The legal algorithm values for each effect type is unique and ultimately determined by the number of branches under the effect type in the system's control tree. A separate branch exists for each algorithm supported by the effect type.

The Program Dump contains an array of values representing the selected algorithms for each effect type.

Another example

Selecting the third Chorus algorithm

Raw data (hex)	Description
F0 06 09 00 01	Basic header: company, product and device id followed by the message class
01 00 00 00	the number of bytes in the message (nibblized)
02 00	the actual data: algorithm 2(nibblized)
02 00 00 00	the number of control levels in the address (2)(nibblized)
00 00 00 00	the first control level (program: 0 or system: 1)(nibblized)
02 00 00 00	the second control level (effect type)(nibblized)
F7	end SysEx

Changing MPX G2 Parameter Values

Changing the parameter values on an MPX G2 involves sending a date message to the system. The message consists of 5 basic components:

SysEx header
Number of data bytes in the message
Actual data
Control Address
End of SysEx

To send a parameter change, there are a few things you need to know:
- What is the control address of the parameter you would like to change?
- How big is the parameter you are changing (in bytes)?
- What is it's legal range of values?

Control Address

There are several ways of determining the control address of a parameter you would like to manipulate:

- Refer to the Control Tree Document:
A complete listing of all parameters in the system and their control address can be found in the control tree text file for the MPX G2 “MPXG2TREE.TXT”.
- Observe the Output:
The MPX G2 can be put into an "Automation" mode in which SysEx data is transmitted whenever parameters are adjusted. The control address of a parameter is sent with each SysEx message so examining the output message reveals the control address. Automation can be enabled from the SYSTEM: MIDI menu.
- Navigate The Tree:
MPXG2 provides enough information (in the form of message classes) to allow an external system to actually navigate the control tree much like you would navigate the directory structure of a computer's hard disk. This requires extensive interaction with the system . While more difficult to implement, this approach allows an external system to, essentially, auto-build an editor.

Parameter Size, Legal Range

The parameter size and legal range of the parameter both contained in the description message for the parameter. This is obtained by requesting Data type, which appears at the control address for the parameter. Once you know the Data Type of the parameter, you request a description of it, which contains its size and legal range. Alternately, you can refer to the control tree text file to get the Data Type number then refer the parameter descriptions text file document for it's size and legal range info.

MPX G2 Parameter Examples

Setting the Chorus Mix to 50%:

Raw data (hex)	Description
F0 06 09 00 01	basic header: company, product and device id followed by the message class
01 00 00 00	the number of bytes in the message (nibblized)
02 03	the actual data: algorithm 1 (mix value)(nibblized)
04 00 00 00	the number of control levels in the address (3)(nibblized)
00 00 00 00	the first control level (program: 0 or system: 1)(nibblized)
01 00 00 00	the second control level (effect type)(nibblized)
01 00 00 00	the third control level (algorithm number)(nibblized)
00 00 00 00	the fourth control level (parameter number number)(nibblized)
F7	end SysEx

Setting the Pitch Level to +3dB:

Raw data (hex)	Description
F0 06 09 00 01	Basic header: company, product and device id followed by the message class
01 00 00 00	the number of bytes in the message (nibblized)
03 00	the actual data: algorithm 1 (Level value)(nibblized)
04 00 00 00	the number of control levels in the address (3)(nibblized)
00 00 00 00	the first control level (program: 0 or system: 1)(nibblized)
00 00 00 00	the second control level (effect type)(nibblized)
01 00 00 00	the third control level (algorithm number)(nibblized)
01 00 00 00	the fourth control level (parameter number)(nibblized)
F7	end SysEx

Tempo Rate Parameter

The tempo on the G2 is always global in the sense that there is only one tempo that affects all elements of the running program, but generally, the tempo is saved and recalled with each program. There is a parameter in the SYSTEM: Mode menu which allows the tempo to be derived from the current "Program" or set as "Global" in which case the tempo is not changed when new programs are loaded.

In either case, the tempo can be set using the following SysEx message (all in hex):

Data (in hex)	Comments
F0 06 09 00 01	Basic header: company, product and device id followed by the message class
01 00 00 00	The number of bytes in the message (nibblized)
04 06 00 00	The actual data: TEMPO OF 100 (nibblized)
03 00 00 00	The number of control levels in the address (3)(nibblized)
00 00 00 00	The first control level (program: 0 or system: 1)(nibblized)
04 01 00 00	The second control level (Tempo parameters)(nibblized)
00 00 00 00	The third control level (Tempo Rate parameter)(nibblized)
F7	End SysEx

The actual tempo (in CAPS) must be nibblized (low nibble first).

MPX G2 - MPX R1 Interface Notes

This provides information on how the MPX G2 interfaces with the MPX R1 MIDI remote.

The interface between the MPX G2 and the MPX R1 consists of both general MIDI messages and SysEx messages. In addition to messages that map directly to controls on the foot controller, there are several SysEx messages that get sent in the background between the two systems.

MPX R1 Detection

On MPX G2 power up, it outputs an "Are you there" handshake message with a device ID of 127 (all devices). If a remote is connected, it responds with an "I’m Alive" message using it’s defined device ID. From this the MPX G2 can determine if a remote is connected and the remote's device ID.

Upon receiving this message, the MPX G2 flashes the message "Remote detected" and displays a custom character ("R1") to the left of the program name in the PROGRAM mode.

In addition, the MPX G2 to the remote telling it various information about its current state sends a message packet.

Once connection is established, the MPX G2 outputs the "Are you there?" message mentioned earlier with the device ID set to match the connected remote every 10 seconds. If still connected, the remote responds with the same "I’m Alive" message above.

If the remote does not send an "I’m Alive" handshake message within 1 second, the message "Remote connection lost" is displayed, the custom character is erased from the PROGRAM mode display and the "Are you there?" is no longer sent. The MPX R1 remote, however, automatically transmits an "I’m Alive" message every 5 seconds even after connection has been lost (cable kicked out, etc.). If the cable is reinstalled, the "I’m Alive" message from the remote notifies the MPX G2 of it’s existence and connection is reestablished.

Program Selection

Program selection is performed using general MIDI program change messages sent from the MPX R1 to the MPX G2. Banks (continuous controllers 0 and 32) are also supported. A bank value of 0 provides access to programs 0-100, 1 provides access to programs 101-200 and 2 provide access to 201-300.

Bypass

The R1 toggles the main and effect bypasses in the G2 using continuous controllers. Though these can be modified in both the R1 and the G2, the default controllers are as follows:

What to bypass	Continuous Controller # (decimal)
Gain	40
FX 1	43
FX 2	47
Chorus	41
Delay	42
Reverb	44
EQ	45
Insert	46
Mstr (main) Bypass	50

TAP

TAP is sent from the MPX R1 to the MPX G2 as continuous controller #119. The MPX G2 recognizes the controller value as a TAP event whenever the value changes from 127. In other words, the controller must go to 127 then change to some other value to be recognized as a TAP event.

A/B

The trigger from the R1 to the G2 to toggle the A/B is a SysEx data message for a panel button with a value of **45 hex** (05 04 nibblized). The message is as follows:

header	Number of data bytes	data (button#)	Num. Control levels
F0 06 09 00 01	01 00 00 00	05 04	03 00 00 00

Level A (System)	Level B (Panel)	Level C (Pnl Button)	End of SysEx
01 00 00 00	08 00 00 00	00 00 00 00	F7

Note that the data (button #) is nibblized (low nibble first, high nibble second).

Tuner

The MPX R1 puts the G2 into the Tuner mode by sending it a SysEx Data message for a panel button with a value of **20 hex** (BYPASS_HOLD)(00 02 nibblized). This is essentially the same as holding down the BYPASS button on the G2 front panel. The message is as follows:

header	Number of data bytes	data (button#)	Num. Control levels
F0 06 09 00 01	01 00 00 00	00 02	03 00 00 00

Level A (System)	Level B (Panel)	Level C (Pnl Button)	End of SysEx
01 00 00 00	08 00 00 00	00 00 00 00	F7

Note that the R1 sends the same message with a value of **36 hex** (BYPASS_RELEASE) (06 03 nibblized) immediately following the "hold" message to simulate a front panel button hold event. The Tuner mode is exited by sending another BYPASS_RELEASE (36 hex) message.

Note that the data (button #) is nibblized (low nibble first, high nibble second).

Pedal

The pedal on the R1 remote transmits data via continuous controller #48 by default (the controller # can be changed in the R1). Several of G2 effects have built in patches to controller #48 ("Pedal Vol" , etc..). When the G2 does not detect an R1 connected, it copies the current value of the rear panel pedal to it's controller 48 location so that the rear panel pedal effectively replaces the pedal on the R1. Note that if the pedal on the remote is set for any controller but 48, it will not control the effects with the built in patches.

Toe Switch

Changes in the Toe Switch from the MPX R1 to the G2 are sent via a SysEx Data message for a panel button with a value of 46 hex (06 04 nibblized) for "Toe Off" and 47 hex (07 04 nibblized) for "Toe On". The message for "Toe On" is as follows:

header	Number of data bytes	data (button#)	Num. Control levels
F0 06 09 00 01	01 00 00 00	07 04	03 00 00 00

Level A (System)	Level B (Panel)	Level C (Pnl Button)	End of SysEx
01 00 00 00	08 00 00 00	00 00 00 00	F7

"Toe Off" uses the same message except that the "data" is 46 hex.

Note that the data (button #) is nibblized (low nibble first, high nibble second).

MPX G2 - Dumps

The product is the ability to move different kinds of data in and out of the box using a single message type. Other products have unique message classes defined for moving structures and tables in and out of the system, but the MPX G2 has folded all of these into Data messages.

The following sections provide detailed information about several special parameter types in the MPX G2, which are typically referred to as "dumps". They provide relatively large blocks of data in a single message. Editor and/or controller programs can use this information to perform off-line editing of these dumps or to provide "virtual" front panels for the system.

MPX G2 - Program Dump

(Type 0x01DF)

In the MPX G2, all of the parameters in the product are divided into 2 groups: "Program" parameters and "System" parameters. A "Program" in the MPX G2 refers to a collection of parameters that are stored and recalled as a group to make sometimes-radical changes to how the system operates. Generally, parameters that end up in the Program camp are those that are changed frequently. All parameters that are not stored in Programs are considered System parameters.

The top level of the control tree for the MPX G2 contains two branches, which correspond to these groups: the "Program" branch and the "System" branch. All parameters that exist under the Program branch affect the active Program running in the G2. All changes to these parameters are stored in a collection of data referred to as an MPXG2_PROGRAM. The "C" language *structure* for this collection as follows:

```
typedef struct
{
PARAM_DATA param_data;
SORT_FLAGS sort_flags;
EFFECT_BLOCK routing_blocks[NUM_CONFIGURABLE_BLOCKS];
TOE_PATCHES toe_patches;
UINT8 alg_nums[NUM_EFFECT_TYPES];
char name[PROG_NAME_SIZE];
UINT8 effects_status;
SOFT_ROW soft_rows[NUM_SOFT_ROW_PARAMS];
TEMPO_DATA tempo_data;
UINT8 unused;
PATCH patches[NUM_PATCHES_PER_PROGRAM];
KNOB_DATA knob;
LFO_DATA lfo_1;
LFO_DATA lfo_2;
RANDOM_GEN_DATA random_gen;
AB_GEN_DATA ab_gen;
ENVELOPE_DATA envelope;
NOISE_GATE noise_gate;
UINT8 bypass_state;
SPEAKER_SIM speaker_sim;
POST_DATA post;
SEND_DATA send;
}MPXG2_PROGRAM;
```

When Programs are transferred to and from the MPX G2, the data is organized using the MPXG2_PROGRAM structure but is generally referred to as "dumps".

In the MPX G2, Program dumps are treated like any other parameter in the system (Mix, Size, etc.). They are transferred to and from the box using the standard data message and always have a control address associated with them. The "data" contained in a Data Message for a Mix parameter is the current value of the Mix while the data contained in a Data Message for a program dump is the program. The number of data bytes contained in a Mix message is 1 while the number of data bytes in a program dump is several hundred. The number of data bytes in the message is different but the message itself is the same. This contrasts sharply with the MPX 1 and most other (older) Lexicon products, which use special SysEx message types for dumps. The MPX G2 uses a totally generic approach for moving all types of data to and from the box; no special messages are used.

While all Programs can be read from the system, only *user* Programs and the *active* program can be sent to the MPX G2. Preset Programs are numbered 1-250 while user Programs are numbered 251-300. Internally, the MPX G2 maps these programs specific locations on its control tree. Because the G2 only uses 8 bit values for control addresses internally, the programs are arranged in-groups of 100 as follows:

# Levels	Level A	Level B	Level C	Level D	Description
2	0x01	0x0A	--	--	General "Program" branch under "System"
3	0x01	0x0A	0x00	--	"Program B1": programs 1-100
4	0x01	0x0A	0x00	0x00	Program 1
4	0x01	0x0A	0x00	0x01	Program 2
4	0x01	0x0A	0x00	0x02	Program 3
...	.	.	..	..	..
4	0x01	0x0A	0x00	0x63	Program 100
3	0x01	0x0A	0x01	--	"Program B2": programs 101-200
4	0x01	0x0A	0x01	0x00	Program 101
4	0x01	0x0A	0x01	0x01	Program 102
..	..	..	..	..	..
4	0x01	0x0A	0x01	0x63	program 200
3	0x01	0x0A	0x02	--	"Program B1": programs 201-300 plus the active program
4	0x01	0x0A	0x02	0x00	Program 201
4	0x01	0x0A	0x02	0x01	Program 202
..	..	..	..	..	..
4	0x01	0x0A	0x02	0x63	Program 300
4	0x01	0x0A	0x02	0x64	Active Program

In dumps to and from the box, the control addresses (Levels, etc..) are 16 bit.

The "active" program (the one currently running in the system) is identified as program number 0xFFFF.

The following table breaks down the program dump byte by byte:

Byte #	Size (Bytes)	Description
0-223	224	Effect Parameter Data
224-226	3	Sort flags
227-272	4	Audio Routing
273	1	FX 1 Algorithm number
274	1	FX 2 Algorithm number
275	1	Chorus Algorithm number
276	1	Delay Algorithm number
277	1	Reverb Algorithm number
278	1	EQ Algorithm number
279	1	Gain Algorithm number
280-291	12	Program name
292	1	Effect Status
293-312	20	Soft Row
313-314	2	Tempo
315	1	Tempo Source
316	1	Beat Value
317	1	Tap Source
318	1	Tap Average
319	1	Tap Source Level
320	1	Unused
321-380	60	Patch data
381-392	12	Knob data
393-400	8	LFO 1 data
401-408	8	LFO 2 data
409-412	4	Random generator data
413-417	5	AB data
418-421	4	Envelope gen
422-433	12	Noise Gate
434	1	Bypass State
435, 436	2	Speaker Simulator
437-439	3	Post
440-442	3	Send

Control Tree Paths

This document contains control tree paths, which define the control address for the parameter, associated with the various fields in the program dump. These paths are very important as they provide information about how the parameters relate to the system as well as specific information about the parameters (name, number of bytes, min/max values, display units, etc.).

Lexicon recommends identifying parameters by their path (control address) instead of their Object Type ID. Though the Object Type ID number is the direct index to the description of the parameter, it does not provide a road map to the location of the parameter in the MPX G2. That's what the control tree path is for. If you want to change a parameter value in the MPX G2, you need it's control tree path. You can derive a parameter's Object Type ID from its control tree path (using the Object Type ID message) but not vice-versa. The description of a parameter should be obtained by using it's control tree path to obtain it's Object Type ID number which is then used to obtain it's description using the Object description message.

The control tree path is also important for interpreting SysEx messages from the MPX G2. When the SYSTEM: MIDI Automation mode is turned on in the MPX G2, any changes to parameters from the front panel of the product result in the transmission of a SysEx message for that parameter. That SysEx message contains a control tree path for the parameter that can be used to map it to local copies of the program dump (update your local program when edits are made from the G2 front panel) as well as dialogs or virtual control panels for the G2.

Throughout this document, control tree paths are defined using the following convention:

L:xxxx A:xxxx B:xxxx C:xxxx D:xxxx E:xxxx

Where the *x* represent hex numbers, the "L" indicates the number of control levels in the control address (path), and the letters A-E represent the control levels with the associate number indicating the value of the level. For example:

L:0003 A:0000 B:0003 C:000A

indicates that the control tree path contains 3 levels, the first with a value of 0, the second with value of 3 and the fourth with a value of 10 (A in hex).

Parameter Data

The parameter data contains parameter information for each of the effect blocks in the system. Each effect block is allocated 32 bytes for parameter data, which multiplied by 7 effect blocks, gets us the 224 bytes. In C this is defined as follows:

```
typedef struct
{
UINT8 data[NUM_EFFECT_TYPES][MAX_PARAM_BYTES];
}PARAM_DATA;
```

But the actual memory looks like this:

Bytes	Effect Block
0-31	FX 1 Effect
32-63	FX 2 Effect
64-95	Chorus Effect
96-127	Delay Effect
128-159	Reverb Effect
160-191	EQ Effect
192-223	Gain Effect

Because each effect block has a variety of *algorithms* that can be loaded into it, the current algorithm assigned to the block must be checked to interpret the parameter data. The FX 1 effect block can be a detuner with 4 parameters or a dual pitch shifter with 5, for instance. Both are "FX 1" effects but one has more parameters. To make matters worse, the third and fourth parameters for the detuner are 1-byte parameters while the third and fourth parameters for the pitch shifter are 2 byte parameters.

Sort Flags

The "Sort Flags" define which sort groups (database) to which the program will be assigned. The system supports 24 different sort categories which are represented as bits in the Sort Flag's two data fields: "Effect Types" (2 bytes) and "Guitar Style" (1 byte). Each bit represents a different sort category. The Effect Type bits are assigned as follows:

Effect Types

Bit #	Mask	Category
0	0x0001	Chorus
1	0x0002	Delay
2	0x0004	Distortion
3	0x0008	EQ
4	0x0010	Flanger
5	0x0020	Gain
6	0x0040	Mod
7	0x0080	Overdrive
8	0x0100	Phaser
9	0x0200	Pitch
10	0x0400	Reverb
11	0x0800	Speaker Simulator
12	0x1000	Wah
13	0x2000	Pre/Post App Type
14	0x4000	Stand Alone App Type
15	0x8000	Inline App Type

Guitar Style

Bit #	Mask	Category
0	0x01	--
1	0x02	Acoustic
2	0x04	Bass
3	0x08	Blues
4	0x10	Clean
5	0x20	Country
6	0x40	Jazz
7	0x80	Rock

The control tree path for this parameter (DBase Flag) is: L:0003 A:0000 B:0011 C:0000

Audio Routing

The Audio Routing section of the program data defines how the audio data moves in and around the system for the current program.

The audio routing is broken up into 9 blocks; one for each effect type, one for the INPUT to the system and one for the OUTPUT. The MPX G2 allows audio moving through the system to be split into (up to) two signal paths referred to as the Upper and Lower paths. Each effect can simultaneously output to two blocks as well as take input from two blocks. In addition, each block supports 2 (stereo) inputs and outputs which can be configured as a single channel, stereo or a single channel summed to mono. All of these aspects of the signal routing are configurable and stored in a program.

In the program dump, routing data is organized as an array of EFFECT_BLOCK structures. The following table breaks out the data fields of the EFFECT_BLOCK with a description of each.

Field	Description
Effect	This is the “Effect Type” for the particular block.
upper_input_conn	This defines the connector used for this block if the "routing" field places it in the upper signal path.
lower_input_conn	This defines the connector used for this block if the "routing" field places it in the lower signal path.
Routing	This defines how this block deals with the upper lower audio paths.
path_type	This defines if the block is on a single path (0) or double path.

Because the blocks represent signal flow through the box, the "effect" type of the first block will always be INPUT and the "effect" type of the last block will always be OUTPUT. Aside from that, the effects can be in any order with the condition that all of the effect types must always be used and no single effect type can be used twice. Effect blocks that have no algorithm loaded still have an EFFECT_BLOCK. The "alg_nums" field of the program must be examined to determine if an algorithm is loaded.

Input Connection Notes

- If the block does not use the lower path, the input connection will be ignored but will become active if needed.
- The INPUT block input connections are only included for consistency and are not actually used by the system.

Routing Notes

The following table outlines the available values for the "routing" field along with a brief description of each.

Value	Name	Description
0	Upper	The effect is on the upper signal path only.
1	Lower	The effect is on the lower signal path only.
2	Merge	The signal from the upper and lower paths is summed and fed into the effect. The output results in a single path.
3	Split	A single (upper) path outputs to the upper and lower paths simultaneously.

Rules and Notes

In addition to the basic range limits that should be placed on the values of each field of the Audio Routing blocks, there are some rules which should be applied when making changes. The basic objectives are to avoid illegal configurations (audio path is broken, etc.); to avoid redundant paths (parallel followed by a parallel, etc.) and to avoid confusing configurations (will actually pass audio but won't look right on the display, etc.).

1. For every split, there must be a merge. (Think of them as open and close parenthesis marks.)
2. The input block must either be upper or split. (The user gets to choose)
3. The output block will be automatically set to upper or merge. (This is determined by the state of the path prior to the output block. If it has been split, then the output block is automatically set to merge. If it is single, the output block is upper.
4. A block is on a single path unless it is in between a split and merge.
5. Split changes a single path into a double path. Merge changes a double path back into a single path.
6. If a split is inserted into a single path and there is no merge between it and the output block, the output block is automatically set to merge.
7. If a merge is inserted into a double path and there is no split between it and the output block, then all following blocks are set to upper.
   If there is a split between the merge and the output block, then all blocks between the merge and the split are set to upper.
8. When the routing configuration of a block is changed its input and output connections should be reset to the default value, stereo.

Toe Patches

The Toe Patches allow multiple effects blocks to be bypassed and unbypassed by pressing the pedal on the R1 remote fully forward. The R1 sends a Panel Button SysEx message to the MPX G2 which checks the TOE_PATCHES struct for assignments and toggles effect bypasses accordingly. The following values are available:

Value	Mode
0	disabled (ignore)
1	Toe OFF bypasses effect Toe ON unbypasses effect
2	Toe ON bypasses effect Toe OFF unbypasses effect

The Toe Patches can be adjusted as parameters using the following control addresses:

Name	Control Tree Path
FX 1 Toe Patch	L:0003 A:0000 B:0015 C:0000
FX 2 Toe Patch	L:0003 A:0000 B:0015 C:0001
Chorus Toe Patch	L:0003 A:0000 B:0015 C:0002
Delay Toe Patch	L:0003 A:0000 B:0015 C:0003
Reverb Toe Patch	L:0003 A:0000 B:0015 C:0004
EQ Toe Patch	L:0003 A:0000 B:0015 C:0005
Gain Toe Patch	L:0003 A:0000 B:0015 C:0005

Algorithm Numbers

The Algorithm Numbers define which algorithm is currently assigned to each of the seven effects blocks in the system. The algorithm numbers appear in the following order:

Index	Effect	Range	Control Tree Path
0	FX 1	0-10	L:0002 A:0000 B:0000
1	FX 2	0-11	L:0002 A:0000 B:0001
2	Chorus	0-18	L:0002 A:0000 B:0002
3	Delay	0-8	L:0002 A:0000 B:0003
4	Reverb	0-5	L:0002 A:0000 B:0004
5	EQ	0-8	L:0002 A:0000 B:0005
6	Gain		L:0002 A:0000 B:0006

The Algorithm Numbers end up defining the "C" level for the control tree path to the particular algorithms when added to the paths in the previous table. The parameters for a particular algorithm would use "D" levels of 0-n depending on the number of parameters used by the algorithm.

A value of zero (0) in all cases represents the "no effect" algorithm assigned. The "no effect" algorithm actually contains a "Mix" and "Level" parameter though they are ignored by the system.

When changing the algorithm number of an effect, remember that the parameter values for the old algorithm may not be compatible with the new algorithm. In much the same way you would reference the "Object Type ID" numbers for a given algorithm when interpreting the 32 byte parameter effect data, you should check the values of all parameters against the limits in the "Parameter Descriptions". If a particular parameter has an out of range value when it is loaded into the MPX G2, the system will fix the value but the edit indicator will always come on when the program is loaded. Note that programs are recognized as "cleared" by the system if the pitch algorithm number is 0xFF.

Program name

This 12 bytes represents the name that will be associated with the stored program. The name does NOT need to be null (0) terminated but **must** instead be padded with space (0x20).

The control tree path for the parameter (Prog Name) associated with this field is: L:0003 A:0000 B:0011 C:0005

Effect Status

This one byte determines the "bypassed/not-bypassed" state of each of the effect blocks. Each of the first six bits is assigned to an effect block with a 0 indicating "bypassed" and a 1 indicating "not-bypassed" (or active).

The control tree path for the parameter (DBase Flag) associated with this field is: L:0003 A:0000 B:0011 C:0001

Soft Row

The Soft Row allows select parameters to be placed in a single convenient menu. Because the available effects may be different for any given program, the legal parameters that can go into the Soft Row is a moving target. In the program dump, the effect type and parameter number are stored. The parameters for the Soft Row, however, are a single index of varying range depending on the currently loaded algorithms. Inside the G2, the index is applied to a dynamically created list of available parameters. The following parameters define which parameters appear in the menu:

Param#	Control Tree Path
0	L:0004 A:0000 B:000E C:0000 D:0000
1	L:0004 A:0000 B:000E C:0001 D:0000
2	L:0004 A:0000 B:000E C:0002 D:0000
3	L:0004 A:0000 B:000E C:0003 D:0000
4	L:0004 A:0000 B:000E C:0004 D:0000
5	L:0004 A:0000 B:000E C:0005 D:0000
6	L:0004 A:0000 B:000E C:0006 D:0000
7	L:0004 A:0000 B:000E C:0007 D:0000
8	L:0004 A:0000 B:000E C:0008 D:0000
9	L:0004 A:0000 B:000E C:0009 D:0000

Note that the "C" level increments while the "D" level stays at 0. Each Soft Row parameter is placed in a separate directory containing a single parameter. The Object Type at each of these locations is the same with the address defining which Soft Row parameter is being modified.

The max value for the Soft Row parameters must be determined dynamically by scanning the Program branch of the control tree, determining which effect branch to include based on algorithms defined in the programs alg_nums field and the SOFT_ROW_ASSIGNABLE flag on the parameters contained therein.

The Soft Row section of the program dump can also be modified by other means as long as the effects and parameter indexes are valid.

The effect/controller type is one of the following:

Values	Effect/Controller	Parameter Indexes
0	FX 1	depends on algorithm
1	FX 2	depends on algorithm
2	Chorus	depends on algorithm
3	Delay	depends on algorithm
4	Reverb	depends on algorithm
5	EQ	depends on algorithm
6	Gain	depends on algorithm
7	Knob	0-3
8	LFO 1	0-6
9	LFO 2	0-6
10	Randomizer	0-2
11	AB Controller	0-4
12	Envelope Generator	0-3

The parameter "index" is the parameter number for the defined effect or controller number that you would like to place in the Soft Values Edit. This number is an index into the parameters for the effect or controller NOT the parameter "Type" number. The "Mix" parameter, for all effects, for instance, is parameter *number* 0. "Level" is ALWAYS parameter number 1. These indexes directly map to the last number of the control address for each parameter.

Tempo

The Tempo subsystem in the MPX G2 allows a wide variety of parameters and events to be perfectly synchronized to a specific tempo (BPM) stored with the program or derived from an incoming MIDI clock. Internally generated tempo can be simply "dialed in" but it can also be tapped in real time. Parameters below determine how the system receives TAPs as well as how the TAPs are interpreted.

Name	Description	Control Tree Path
Rate	The current tempo stored with the program. Must be 41-400 BPM.	L:0003 A:0000 B:0014 C:0000
Source	Determines if the tempo is derived from the Rate or from incoming MIDI clock.	L:0003 A:0000 B:0014 C:0001
Beat Value	Determines how presses of the TAP button are interpreted.	L:0003 A:0000 B:0014 C:0002
Tap Source	Allows controller sources to be used for TAP.	L:0003 A:0000 B:0014 C:0003
Tap Average	Allows tempo to be derived from a series of TAPs instead of a single pair of presses. Improves precision.	L:0003 A:0000 B:0014 C:0004
Source Level	Sets the threshold for the Tap Source.	L:0003 A:0000 B:0014 C:0005

Patch system data

The Patching system in the MPX G2 allows both internal and external control sources to control program parameters. Each program contains 5 Patches which each takes a single control source to control a single parameter. In the program dump, data for the Patches are organized as an array of PATCH structures. All of the fields in the PATCH structures are accessible via parameters as follows: (control tree address for patches 1 shown)

Name	Description	Control Tree Path
Source	The internal control source or MIDI controller that drives the patch	L:0004 A:0000 B:000D C:0000 D:0000
Source min	The value of the source control's data that drives the Destination to its min value.	L:0004 A:0000 B:000D C:0000 D:0001
Source mid	The value of the source control's data that drives the Destination to it's mid value.	L:0004 A:0000 B:000D C:0000 D:0002
Source max	The value of the source control's data that drives the Destination to it's max value.	L:0004 A:0000 B:000D C:0000 D:0003
Destination Effect	This is the effect block that is being controlled by the patch.	see notes below
Destination parameter	The is the particular parameter for the effect that is being controlled by the patch.	see notes below
Destination min	This is the minimum value that the patch can drive the parameter to. The parameter is set for this value when the Source is at its "min" value.	L:0004 A:0000 B:000D C:0000 D:0005
Destination mid	The parameter is set for this value when the Source is at it's "mid" value.	L:0004 A:0000 B:000D C:0000 D:0006
Destination max	This is the maximum value that the patch can drive the parameter to. The parameter is set for this value when the Source is at its "max" value.	L:0004 A:0000 B:000D C:0000 D:0007

To control the parameters for the other patches, control level "C" must be incremented; 1 for patch 2, 2 for patch 3, etc.

Patch "Source" Notes:

Patch Sources are not simply MIDI controller numbers. Available controllers include internally generated signals as well as incoming MIDI data.

Patch "Destination Effect and Parameter" Notes:

The patch destination "effect" and "parameter" are not directly accessible via a control address. On the G2, the destination parameter is selected from a single list of available parameters that is dynamically constructed for the current program to include only parameters that are currently available (you cannot patch to a parameter on an effect that is not loaded). In addition to the parameters being currently active, they also need to be flagged as "Patchable". Each parameter's description contains a bitmapped byte of control flags, which identify various attributes of the parameter including its ability to be controlled by a patch.

Only parameters that are stored in programs can be the destination/target for a patch. The "Destination *Effect*" and "Destination *Parameter*" each ultimately map to a Control Tree Path for the parameters being controlled. The Destination *Effect* maps directly to the control level B of the MPX G2's control tree under the Program branch; you use the value of the control level for the Effect. Though it's name doesn't suggest it, the Effect can be any of the B level branches under Program including LFOs, Speaker Sim, Noise Gate and Post controls. The Destination *Parameter* maps directly to the control level D values for real effects (Gain, Reverb, etc.) and control level C for all others (everything following Gain).

When controlling the MPX G2 via MIDI, there is a single continuous parameter that the G2 uses to define the Destination Effect and Parameter for each program Patch. The following table outlines the control addresses for these parameters:

Patch #	Control Tree Path
1	L:0004 A:0000 B:000D C:0000 D:0004
2	L:0004 A:0000 B:000D C:0001 D:0004
3	L:0004 A:0000 B:000D C:0002 D:0004
4	L:0004 A:0000 B:000D C:0003 D:0004
5	L:0004 A:0000 B:000D C:0004 D:0004

When queried for the descriptions of these parameters (objects), the maximum legal value will change depending on what effects are loaded. When the MPX G2 detects a remote control connected to the system (R1 or a controller program), it automatically transmits the updated descriptions of these parameters. If a controlling program updates its local copy of the parameter description, the parameter will automatically have the correct range. The remote is detected via handshake messages (see the MPX R1 Interface Notes section).

For both the Destination Effect and Parameter fields, a value of 0xFF represents "unassigned".

Patch Destination Min, Mid and Max

The Patch Destination Min, Mid and Max present an interesting problem. Because the Destination parameter is selectable, the actual legal values which can be used for the Min, Mid and Max change with the Destination parameter. The Patch Destination Min, Mid and Max parameters are what we refer to as Dynamic parameters; their descriptions are not fixed. The Destination Min, Mid and Max parameters always match the parameter defined as the Destination however.

When queried for the Object Type ID at these locations, the G2 will report the ID of the currently selected Destination parameter. If a remote (R1 or controller program) is detected, the MPX G2 automatically transmits the updated IDs when the destination is changed.

Knob Data

The "Knob" (also referred to as the "custom controller") effectively allows the user to rename a parameter and place it in the list of "Soft Row" parameters. The Knob is really just an internal control source that can be used with program patches. The Knob has a programmable min and max values as well as a name that appears when the Knob is assigned to the Soft Row. The parameters are as follows:

Name	Description	Control Tree Path
Value	Represents the current value of the Knob.	L:0003 A:0000 B:0007 C:0000
Low	Sets minimum value selectable via the Knob.	L:0003 A:0000 B:0007 C:0001
High	Sets the maximum value selectable via the Knob.	L:0003 A:0000 B:0007 C:0002
Name	Sets the name that appears in the Soft Row if the Knob is used.	L:0003 A:0000 B:0007 C:0003

The minimum and maximum values are 8 bit values but should be limited to 0-127 like all other controllers.

LFOs

The MPX G2 contains two LFOs (low frequency oscillators) which provide modulating control sources for program patches. Each LFO has 4 outputs, which appear, as sources for program patches:

Patch Source Name	Description
Puls1/Puls2	This is a square wave output.
Tri1/Tri2	Triangle wave output
Sine1/Sine2	Sinewave output
Cos1/Cos2	Cosine wave output

Each program dump contains parameter data, which defines the operating mode of the LFOs. The following table lists each of the LFO parameters in a program dump along with their control tree paths.

LFO 1 Parameters

Name	Description	Control Tree Path
Mode	Defines the operating mode of the LFO.	L:0003 A:0000 B:0008 C:0000
Rate	Defines the modulation rate of the LFO	L:0003 A:0000 B:0008 C:0001
PW	Defines the pulse width of the LFO	L:0003 A:0000 B:0008 C:0002
Phase	Defines the phase of the LFO	L:0003 A:0000 B:0008 C:0003
Depth	Defines the depth of the LFO	L:0003 A:0000 B:0008 C:0004
OnLvl	Defines the trigger level of the source	L:0003 A:0000 B:0008 C:0005
OnSrc	Defines the control source for the LFO trigger	L:0003 A:0000 B:0008 C:0006

LFO 2 Parameters

Name	Description	Control Tree Path
Mode	Defines the operating mode of the LFO.	L:0003 A:0000 B:0009 C:0000
Rate	Defines the modulation rate of the LFO	L:0003 A:0000 B:0009 C:0001
PW	Defines the pulse width of the LFO.	L:0003 A:0000 B:0009 C:0002
Phase	Defines the phase of the LFO	L:0003 A:0000 B:0009 C:0003
Depth	Defines the depth of the LFO	L:0003 A:0000 B:0009 C:0004
OnLvl	Defines the trigger level of the source	L:0003 A:0000 B:0009 C:0005
OnSrc	Defines the control source for the LFO trigger	L:0003 A:0000 B:0009 C:0006

Random Number Generator

The Random Number Generator produces random numbers that can be used as control sources in program patches. The parameters for the generator are as follows:

Name	Description	Control Tree Path
RndLo	Defines the minimum number generated.	L:0003 A:0000 B:000A C:0000
RndHi	Defines the maximum number generated.	L:0003 A:0000 B:000A C:0001
Rate	Defines the rate at which the number changes.	L:0003 A:0000 B:000A C:0002

AB Generator

The AB Generator produces ramping numbers that can be used as control sources in program patches. The parameters for the AB generator are as follows:

Name	Description	Control Tree Path
Mode	Defines the operating mode of the generator.	L:0003 A:0000 B:000B C:0000
ARate	Sets the rate of the change during the transition from 0 to 127. Higher values increase the speed.	L:0003 A:0000 B:000B C:0001
BRate	Sets the rate of the change during the transition from 127 to 0. Higher values increase the speed.	L:0003 A:0000 B:000B C:0002
OnLvl	Sets the threshold for OnSrc triggering.	L:0003 A:0000 B:000B C:0003
OnSrc	Selects a controller to trigger the AB generator.	L:0003 A:0000 B:000B C:0004

Envelope Generator

The Envelope Generator produces numbers that can be used as control sources in program patches based on the signal level of the sources. The parameters for the generator are as follows:

Name	Description	Control Tree Path
Src1	Selects a signal source that will feed the generator.	L:0003 A:0000 B:000C C:0000
Src2	Selects a second signal source that will feed the generator.	L:0003 A:0000 B:000C C:0001
ATrim	Defines the rate at which the envelope generator initially responds to signal from the sources.	L:0003 A:0000 B:000C C:0002
Resp	Defines the rate at which the envelope releases (returns to 0) when source signal reduces.	L:0003 A:0000 B:000C C:0002

Noise Gate

The Noise Gate blocks signal from being output at the Send jack and/or from feeding the "Post" signal path (anything after the Gain block in the routing diagram). The parameters for the Noise Gate are as follows:

Name	Description	Control Tree Path
Enable	Off/Guitar Input/Returns Only.	L:0003 A:0000 B:0013 C:0000
Send	Off/On – analog noise gate at "send" output jack.	L:0003 A:0000 B:0013 C:0001
Thrsh	83 to 0dB - the level above which a signal will open the gate.	L:0003 A:0000 B:0013 C:0002
Atten	-90 to 0dB - the atten level of the digital noise gate in the "post distortion" block.	L:0003 A:0000 B:0013 C:0003
Offset	Off to -12dB - adjusts the size of a "hysteresis band" in the detector.	L:0003 A:0000 B:0013 C:0004
ATime	Attack, 0 to 2000 – milliseconds for the opening of the gate.	L:0003 A:0000 B:0013 C:0005
HTime	Hold, 20 to 500 milliseconds - a pause between detection of a noise gate close and the actual closing of the gate	L:0003 A:0000 B:0013 C:0006
RTime	Release, 0 to 2000 – milliseconds for the closing of the gate (ramp down time).	L:0003 A:0000 B:0013 C:0007
Delay	0 to 10 - milliseconds of gate "look ahead"	L:0003 A:0000 B:0013 C:0008

Delay	0 to 10 - milliseconds of gate "look ahead".	L:0003 A:0000 B:0013 C:000

The option under the "Thresh" parameter allows you to select Guitar In or Returns (from the jacks, plus the speaker sim) so this is actually user controlled.

Bypass State

This field defines what the bypass state of the system will be when the program loads. A value of 0 causes the program to load with the bypass OFF while a value of 1 causes the program to load with the system bypassed. A parameter was not defined for this field but the Bypass State of the MPX G2 can be toggled on and off using the assigned Bypass Controller. The controller selected for system bypass is at the following control tree path:
L:0004 A:0001 B:0004 C:0000 D:0000

Speaker Simulator

The Speaker Simulator adds filtering to the audio feeding the "Post" signal path (anything after the Gain block in the routing diagram) in situations where a physical speaker is not present. The parameters for the Speaker Simulator are as follows:

Name	Description	Control Tree Path
Enable	Turns the speaker sim on or off.	L:0003 A:0000 B:0012 C:0000
Cabinet	Selects which speaker cabinet is being emulated.	L:0003 A:0000 B:0012 C:0001

Post

The Post parameters set the mix and signal levels of audio through the "Post" signal path (anything after the Gain block in the routing diagram). The parameters are as follows:

Name	Description	Control Tree Path
Mix	Balances between the amount of dry and wet signal.	L:0003 A:0000 B:000F C:0000
Level	Sets the unbypassed signal level out of the Post signal path.	L:0003 A:0000 B:000F C:0001
PostBypLvl	Sets the bypassed signal level out of the Post signal path.	L:0003 A:0000 B:000F C:0002

Send

The Send parameters set the signal levels of audio through the "Send" signal path (everything before and including the Gain block in the routing diagram). The parameters are as follows:

Name	Description	Control Tree Path
Level	Sets the unbypassed signal level out of the Send jack.	L:0003 A:0000 B:0010 C:0000
SendBypLvl	Sets the bypassed signal level out of the Send jack.	L:0003 A:0000 B:0010 C:0001

MPX G2 - All LEDs Dump

(Type 0x0165)

This dump contains the current contents of the led buffer in the MPX G2 that controls the states of the front panel LEDs. The actual front panel LEDs is represented as individual bits in the dump. These bits can be monitored to implement a virtual MPX G2 panel. Note that the MPX G2 also supports a special mode in which it automatically transmits the All LED Dump whenever the state of an LED in the system changes. The following table outlines the assignments of the bytes/bits in the dump:

Byte #	Description	Bit Assignments	"On" State
0	unused	-	-
1	Column 1	bit 0 = Scan bit 0 (1) (see note below) bit 1 = Scan bit 1 (0) bit 2 = Scan bit 2 (0) bit 3 = Gain LED bit 4 = FX 1 LED bit 5 = FX 2 LED bit 6 = Program LED bit 7 = unused	- - - 0 0 0 0 -
2	unused	-	-
3	Column 2	bit 0 = Scan bit 0 (0) (see note below) bit 1 = Scan bit 1 (1) bit 2 = Scan bit 2 (0) bit 3 = Chorus LED bit 4 = Delay LED bit 5 = Reverb LED bit 6 = Edit LED bit 7 = unused	- - - 0 0 0 0 -
4	7 Segment Display 1 (left)	bit 0 = segment 'a' bit 1 = segment 'b' bit 2 = segment 'c' bit 3 = segment 'd' bit 4 = segment 'e' bit 5 = segment 'f' bit 6 = segment 'g' bit 7 = decimal point	1 1 1 1 1 1 1 1
5	Column 3	bit 0 = Scan bit 0 (1) (see note below) bit 1 = Scan bit 1 (1) bit 2 = Scan bit 2 (0) bit 3 = EQ LED bit 4 = Insert LED bit 5 = Bypass LED bit 6 = System LED bit 7 = unused	- - - 0 0 0 0 -
6	7 Segment Display 2 (middle)	bit 0 = segment 'a' bit 1 = segment 'b' bit 2 = segment 'c' bit 3 = segment 'd' bit 4 = segment 'e' bit 5 = segment 'f' bit 6 = segment 'g' bit 7 = decimal point	1 1 1 1 1 1 1 1
7	Column 4	bit 0 = Scan bit 0 (0) (see note below) bit 1 = Scan bit 1 (0) bit 2 = Scan bit 2 (1) bit 3 = Tempo LED bit 4 = A LED bit 5 = Soft Row LED bit 6 = Store LED bit 7 = unused	- - - 0 0 0 0 -
8	7 Segment Display 3 (right)	bit 0 = segment 'a' bit 1 = segment 'b' bit 2 = segment 'c' bit 3 = segment 'd' bit 4 = segment 'e' bit 5 = segment 'f' bit 6 = segment 'g' bit 7 = decimal point	1 1 1 1 1 1 1 1
9	Column 5	bit 0 = Scan bit 0 (1) (see note below) bit 1 = Scan bit 1 (0) bit 2 = Scan bit 2 (1) bit 3 = MIDI LED bit 4 = B LED bit 5 = Option LED bit 6 = unused bit 7 = unused	- - - 0 0 0 0 -

A value of 1 represents an LED being turned ON and 0 representing the LED turned OFF for the 7 segment displays. For the "Columns" the states are reversed: 0 = on, 1 = off.

Note that the first 3 bits of the "Columns" contain scan information used by the system to drive hardware multiplexed, which provides scanning for both the LEDs and the keyboard. These bits can be ignored.

The 7 segment displays are represented with each bit assigned to a segment. The following diagram shows where each segment appears on the display:

```
                  -----
d0 = segment a   |  d  |
d1 = segment b   |c   e|
d2 = segment c   |  g  |
d3 = segment d   |b   f|
d4 = segment e   |  a  |
d5 = segment f    -----
d6 = segment g
d7 = Decimal Point
```

The number images are defined in the system software as follows:

```
             /* pgfe dcba */
0 IMAGE 0x3f /* 0011 1111 */
1 IMAGE 0x30 /* 0011 0000 */
2 IMAGE 0x5b /* 0101 1011 */
3 IMAGE 0x79 /* 0111 1001 */
4 IMAGE 0x74 /* 0111 0100 */
5 IMAGE 0x6d /* 0110 1101 */
6 IMAGE 0x6f /* 0110 1111 */
7 IMAGE 0x38 /* 0011 1000 */
8 IMAGE 0x7f /* 0111 1111 */
9 IMAGE 0x7c /* 0111 1100 */
P IMAGE 0x5e /* 0101 1110 */
```

Note - Though the description of this parameter indicates it is 32 bytes, only the first 10 bytes are valid. The remaining bytes should be ignored. Due to this discrepancy, this dump should not be sent to the MPX G2 even though the system will allow it

MPX G2 - Display Dump

(Type 0x0154)

The display dump allows the current contents of the display to be extracted or messages to be placed on the display of the connected MPX G2. The dump contains 32 characters; one for each segment of the LCD display(top left to bottom right). All ASCII characters can be used along with the numbers 0-7 which access the 8 custom characters supported by the display. Because of this, NULL terminated strings should not be used. The NULL will be interpreted as the first custom character.

When "requested", the current contents of the display buffer is transmitted.

Please note that several displays in the system use the custom characters. This can be a problem for 'C' language functions which look for null terminated strings. Always parse the strings and deal with the custom characters appropriately.

MPX G2 - Custom Character Bitmap Dumps

(Type 0x0166)

This Dump allows you to send bit-mapped custom characters to the LCD in the MPX G2. There are a total of 8 custom characters available in the LCD which can be used like any other (ASCII) character when writing to the display. Normally, characters are written to the display by sending the ASCII number representing the character to the display. Custom characters are displayed by sending the numbers 0-7 for the custom characters 0-7 (respectively) to the display. The use of the number 0 for the first character is unfortunate because of its implications in the 'C' programming language but it is a fact of life when dealing with these displays.

Outside developers can use this dump to generate their own characters for a flashy start-up screen when their system connects with a MPX G2.

Each custom character is 5 pixels wide by 7 pixels high with one additional row of pixels on the bottom for the underline. When loading custom characters, the data is transferred as 8 bytes of data, each representing one row of the character starting from the top. Each bit of the byte corresponds to one pixel with 1 turning the pixel on and 0 turning it off. Because each character is only 5 pixels wide, the top 3 bits are not used. As a result, all character will either have a 1 or a 0 in the high nibble. For example, the letter 'T' would be represented as follows:

0x1f, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x00

If you look at it in a binary format you can really see the underlying image.

Row	in Binary	in Hex
0	0001 1111	0x1F
1	0000 0100	0x04
2	0000 0100	0x04
3	0000 0100	0x04
4	0000 0100	0x04
5	0000 0100	0x04
6	0000 0100	0x04
7	0000 0000	0x00

You would use the custom characters by sending a display dump message using the custom character numbers in the places where you want the custom characters to appear on the display.

Note that the dump always contains all 8 custom characters for a total of 64 data bytes.

MPX G2 - Setup Dumps

(Type 0x015E)

The system supports 5 stored setups. The dumps can be requested via MIDI SysEx or transmitted from the MPX G2 from the SYSTEM: MIDI menu (Dumps). When sent back to the system, they are stored in the system's memory but are not made active until they are either selected via the SYSTEM:Modes menu (Setup Load) or via a "SysEx Setup Select" message. This is really just a Data message to the "Setup Select" control address. The "data" portion of the message defines which setup will be selected.

The following example would select setup #3

header	Number of data bytes	Data	Num. Control levels
F0 06 0F 00 01	01 00 00 00	02 00	03 00 00 00

Level A (System)	Level B (Audio Params)	Level C (Setup Sel)	End of SysEx
01 00 00 00	01 00 00 00	0D 00 00 00	F7

The contents of the dumps are broken out as follows (in the indicated order):

Byte #	Description	# Bytes	Type#
1,2	MIDI Receive Channel	1	0x0136
3-6	MIDI Transmit Speed	2	0x0137
7,8	MIDI SysEx Receive	1	0x0138
9,10	MIDI Remote	1	-
11,12	MIDI Soft Thru	1	0x017C
13,14	MIDI SysEx On/Off	1	0x015C
15,16	MIDI Transmit	1	0x0139
17-20	Pgm# Offset	2	0x013A
21,22	MIDI Automation	1	0x013B
23,24	MIDI Clock Send On/Off	1	0x013C
25,26	Program Sort Mode	1	0x013D
27,28	Modes Brightness	1	0x013E
29,30	Automation Xmit ID	1	0x013F
31,32	MIDI Pgm Change On/Off	1	0x0140
33,34	MIDI Ctl Smooth	1	0x014F
35,36	Modes MemProtect On/Off	1	0x0142
37,38	Store Prompt On/Off	1	0x0150
39,40	Modes Sleep	1	0x0143
41,42	Modes Mix	1	0x0144
43,44	Modes Pgm Load	1	0x0141
45,46	Pgm+ controller	1	0x0146
47,48	Pgm- controller	1	0x0147
49,50	Audio Soft Sat	1	0x0148
51,52	pedal_max	1	-
53,54	pedal_delta	1	-
55,56	Modes Tempo	1	0x0151
57,58	Modes Pgm Load	1	0x0141
59,60	Modes Send Bypass	1	0x016F
61,62	Modes Post Bypass	1	0x016E
63,64	Insert Modes	1	0x0170
65,66	Speaker Sim Modes	1	0x0172
67,68	Noise Gate Modes	1	0x0173
69,70	Reverb Modes	1	0x0174
71,72	Bypass Modes	1	0x01C6
73,74	Tuner Calibration	1	0x018E
75,76	Tuner Offset	1	0x018F
77,78	Tuner Bypass Modes	1	0x0190
79,80	Audio Output Mode	1	0x011A
81-98	bypass_patch_sources	9	-
99-116	setup_name	9	-

MIDI Receive" Channel

This is the MIDI channel on which the system recognizes incoming data. Legal values are 0 to 15 for the standard midi channels, 16 for MIDI off and 17 for Omni mode.

MIDI Transmit Speed

This sets the rate at which midi data is output from the system. Only the values in the following table should be used in this field:

Fast	0x01
Medium Fast	0x02
Medium Slow	0x04
Slow	0x40

MIDI SysEx Receive

This is the device ID used in all outgoing SysEx dump messages and incoming messages. Legal values are 0 to 126.

MIDI Remote

This is the device ID used in all outgoing messages to the R1 remote. Legal values are 0 to 126.

MIDI Soft Thru

This enables (1) or disables (0) MIDI soft thru mode.

MIDI SysEx On/Off

This enables (1) or disables (0) reception of SYSEX data by the system.

MIDI Transmit

This is the MIDI channel on which the system outputs MIDI data. Legal values are 0 to 15 for the standard midi channels and 16 for MIDI off.

Pgm# Offset

This 2 byte value sets the "Pgm# Offset" on the system which allows incoming standard program change messages to be offset from the first program. Normally a program change value of 0 loads program 1.

MIDI Automation

This determines if the MIDI automation in the system is on (1) or off (0)

MIDI Clock Send On/Off

This determines if MIDI clock is (1) or is not (0) to be transmitted from the system (MIDI out).

Program Sort Mode

This sets the current program sort mode in the system. The following values have been defined:

SORT_BY_NAME	0
SORT_BY_NUMBER	1
SORT_BY_GUITAR_STYLE	2
SORT_BY_FX_TYPE	3
SORT_BY_APP_AND_FX	4
SORT_BY_MIDI_MAPS	5
SORT_BY_MIDI_CHAINS	6
SORT_BY_TOP_10	7

Modes Brightness

This sets the brightness level of the system's fluorescent display. Legal values are 0 to 15.

Automation Xmit ID

This sets the device ID of the outgoing SYSEX data generated by the system's automation. Legal values are 0 to 126 or 127 for all targets. (this is an option of the MIDI Automation On/Off)

MIDI Pgm Change On/Off

This enables (1) or disables (0) program changes via standard MIDI program change messages.

MIDI Ctl Smooth

This sets the amount of interpolation that is performed on the incoming MIDI controller data. 0 = no interpolation, 100 = full interpolation.

Modes MemProtect On/Off

This turns the system's program memory protect mode is turned on (1) or off (0).

Store Prompt On/Off

This sets the auto store mode to on (1) or off (0). (this is an option of Modes MemProtect).

Modes Sleep

This sets the current sleep mode from the following:

NO_SLEEP_MODE	0
HELP_SLEEP_MODE	1

Modes Mix

This sets the current mix mode: 0 = global, 1 = program.

Modes Pgm Load

This defines how the system will route the audio through the system during a program load. Bypass = 0 or all mute = 1.

Pgm+
Pgm-

These define the controllers used to increment and decrement the current program. Refer to the list of controllers in Controller Indexes for legal values.

Audio Soft Sat

This turns the soft saturation circuits on (1) or off (0).

pedal_max
pedal_delta

These are internally generated calibrations of the connected foot pedal. pedal_max is related to the minimum pedal voltage after calibration. It decreases from 0xFF as the pedal minimum voltage increases. (The reason it's backwards, and the reason it's called "max", is that the pedal ADC is basically a down counter).

pedal_delta is proportional to the range of the pedal voltage after calibration (the max voltage - the min voltage). In other words, pedal_max is the offset term and pedal_delta is the scaling term.

Modes Tempo

This determines whether the current tempo is global (0) or program specific (1).

Modes Pgm Load

This sets the auto load mode to Manual (0) or auto-load (1).

Modes Send Bypass

This sets how the audio is routed through the send output when system bypass is turned on. The following values have been defined:

NORMAL_BYPASS	0
ALL_MUTE_BYPASS	1
INPUT_MUTE_BYPASS	2

Modes Post Bypass

This sets how the audio is routed through the post gain section when system bypass is turned on. The following values have been defined:

NONE	0
BYPASS	1
ALL_MUTE_BYPASS	2
INPUT_MUTE_BYPASS	3

Insert Modes

This controls where the inserts are sent in the analog circuitry around the post gain section. The following values have been defined:

FX_LOOP	0
MIX	1
PARALLEL	2

Speaker Sim Modes

This sets the current speaker sim mode: 0 = global, 1 = program.

Noise Gate Modes

This sets the current noise gate mode: 0 = global, 1 = program.

Reverb Modes

This sets the current reverb mode: 0 = global, 1 = program.

Bypass Modes

This sets the current bypass mode: 0 = global, 1 = program.

Tuner Calibration

This sets the current tuner calibration setting. Legal values are 0 to 11.

Tuner Offset

This sets the current tuner offset setting. Legal values are –50 to 50.

Tuner Bypass Modes

This sets the current tuner bypass mode:

MUTE	0
POST_MUTE	1
ALL_BYPASS	2

Audio Output Mode

This sets the configuration of the audio output signal. The following choices have been defined:

STEREO_OUTPUT_MODE	0
MONO_OUTPUT_MODE	1

bypass_patch_sources

This is an array containing the sources which activate bypass for the following 9 blocks:
Mstr, Gain, FX 1, FX 2, Chrs, Dly, Rvb, Eq, and Ins.

setup_name

This is an array containing the ASCII name string for the setup. This can be a maximum of 9 characters and should NOT be null terminated. Unused characters should be spaces (0x20).

MPX G2 - Global Patches Dump

(Type 0x0163)

This message class allows you to send Global Patches dumps to, or receive them from the system. There are 10 global patches in the system, each with a one byte source and destination. The sources are the controllers from the "Controller Indexes" beginning with "Pedal" (0x14) and ending at "Toe" (0x9D). A value of 0x13 is used if the patch is unassigned ("None" displayed).

The destinations are as follows:

Value	Destination
0	none
1	Post Mix
2	FX 1 Mix
3	FX 2 Mix
4	Chorus Mix
5	Delay Mix
6	Reverb Mix
7	EQ Mix
8	Post Level
9	FX 1 Level
10	FX 2 Level
11	Chorus Level
12	Delay Level
13	Reverb Level
14	EQ Level
15*	Loop – Clear
16*	Loop – Layer
17*	Loop – Replace
18*	Loop - Delay
19*	Loop - Start/Stop

* - These commands are only effective if the JamMan effect is loaded in the Delay block.

The dumps are formatted as 20 bytes in 2 ten byte arrays; the first 10 are for the sources (1-10) and the second 10 bytes are the destinations (11-20).

MPX G2 - Bypass Controllers Dump

(Type 0x0162)

This dump allows you to send "Bypass Controllers" assignments to, or receive them from the system. The bypass controllers allow the operator to define a particular controller to toggle the effects and system bypass on and off. The 9 bypass controllers are assigned as follows:

Byte Offset	Target	Default Controller
0	Master Bypass	43
1	Gain	47
2	FX 1	41
3	FX 2	42
4	Chorus	44
5	Delay	45
6	Reverb	40
7	EQ	46
8	Insert	50

The dump consists of 9 one byte controller sources that can be patched to these destinations. The sources are the controllers from the Controller Indexes beginning with "Tog1" (0x15) and ending at "Toe" (0x9D). A value of 0x14 is used if the controller is unassigned ("None" is displayed).

MPX G2 - Remap Controllers Dump

(Type 0x0161)

This dump allows you to send "Remap Controllers table" assignments to, or receive them from the system. The remap controllers table is used by the system to define the MIDI controller number which will be used to transmit the control source if it is used in a patch and has a non-0xFF value in this table. The value 0xFF designates "not assigned". The system supports 158 controller sources, each of which have an associated remap value in the remap table.

The table in the list of "Controller Indexes" identifies the assignment of each byte of this dump sequentially.

MPX G2 - Current Choices Dump

(Type 0x015B)

This message allows you to get the current settings of all menu choices in the system. Typically this dump would be used to initialize the state (bypass, running program, etc.) of the system.

In the system software, the current choices are stored in a data structure of the following type:

Byte #	Description	# Bytes
1,2	program_bank	1
3,4	current_effect_edit_parameter	1
5,6	cur_alg_select	1
7,8	(reserved)	1
9,10	(reserved)	1
11,12	compare_mode	1
13,14	dbase_edit	1
15,16	soft_row	1
17,18	soft_row_edit	1
19,20	current_map_block	1
21,22	cur_routing_item	1
23,24	(reserved)	1
25,26	order	1
27,28	mix	1
29,30	audio	1
31,32	audio_options	1
33,34	midi	1
35,36	chains	1
37,38	chain	1
39,40	chain_link	1
41,42	cur_midi_map	1
43,44	midi_map	1
45,46	midi_map_element	1
47,48	store	1
49,50	edit	1
51,52	system	1
53,54	cur_edit_page	1
55-58	(reserved)	2
59,60	cur_edit_param_index	1
61,62	cur_num_edit_params	1
63,64	cur_system_page	1
65-68	(reserved)	2
69,70	cur_system_param_index	1
71,72	cur_num_system_params	1
73-76	selected_program_num	2
77-80	running_program_num	2
81,82	operating_mode	1
83,84	(reserved)	1
85,86	cur_setup_num	1
87,88	setup	1
89,90	(reserved)	1
91,92	cur_bypass_patch_num	1
93,94	cur_global_patch_num	1
95,96	cur_global_patch_menu	1
97,98	cur_ui_dump_type	1
99,100	cur_cont_remap	1
101,102	patch	1
103,104	cur_patch_num	1
105,106	(reserved)	1
107,108	cur_auto_store	1
109,110	cur_tempo	1
111,112	(reserved)	1
113,114	cur_verify	1
115,116	cur_verify_store	1
117,118	(reserved)	1
119,120	(reserved)	1
121-124	(reserved)	2
125-128	(reserved)	2
129-132	(reserved)	2
133-136	(reserved)	2
137-140	(reserved)	2
141-144	(reserved)	2
145-148	(reserved)	2
149-152	(reserved)	2
153-156	(reserved)	2
157-160	(reserved)	2
161-164	(reserved)	2
165-168	(reserved)	2

program_bank

This is the program bank that is set when controller32 (bank select) is sent to the box. This is used to offset incoming standard program change messages by 0, 100 and 200 when set to 0, 1, and 2 respectively.

current_effect_edit_parameter

This contains the parameter number of the effect edit menu. Note that this is really only significant when in the EDIT mode with effect parameters visible because of an "effect_matrix" that contains the current positions in each effect's menu. The "current_effect_edit_parameter" is only the currently displayed version. The "effect_matrix" is not accessible via MIDI.

cur_alg_select

This contains the current value of the effect (algorithm) select menu. When displayed, this translates into the algorithm number for the selected effect.

compare_mode

This contains the current "compare mode" for the system (1 = on, 0 = off).

dbase_edit

This contains the current choice for the "Database" menu.

soft_row

This contains the current choice for the "Soft Row" menu when accessed via the Edit mode.

soft_row_edit

This contains the current choice for the "Soft Row Edit" menu when accessed via the Program mode.

current_map_block

This contains the current effect block being edited in the "Audio Routing" menu. Note that block number 0 is ALWAYS the input and block number 8 is always the output.

cur_routing_item

This contains the current effect block being edited in the "Audio Routing" menu OPTION. Again, block number 0 is ALWAYS the input and block number 8 is always the output.

order

This contains the current effect block being moved in the "Effect Order" menu.

mix

This contains the current active position in "Mix" menu. (range: 0-13)

audio

This contains the current "Audio" menu item. (range: 0-5)

audio_options

This contains the current "Audio" menu OPTION item. (range: 0-2)

midi

This contains the current "MIDI" menu item. (range: 0-10)

chains

This contains the current "Chains" menu item. (range: 0-2)

chain

The currently selected chain (range: 0-9)

chain_link

The currently selected element in the currently selected chain (range: 0-9)

cur_midi_map

This contains the current choice for the "MIDI Maps" menu. (range: 0-3)

midi_map

This contains the current MIDI map. (range: 0-2)

midi_map_element

This contains the current choice in the current MIDI map. (range: 0-127)

store

This contains the current "Store" menu item. A value of 0 allows the encoder to select a program number while the 1-12 allow the encoder to edit the program's name.

edit

This contains the current selected "Edit" menu item.

system

This contains the current selected "System" menu item.

cur_edit_page

This is the current "Edit" menu item (Compare, Meter Assign, etc.) (range: 0-18)

cur_edit_param_index

This is a more global version of the "current choices" for menus under "Edit". In some menus, this actually replaces the "current choice" variable. The range varies depending on which menu is active.

cur_num_edit_params

This is global "Edit" mode count of the max value for the active menu.

cur_system_page

This is the current "System" menu item (Audio, Mode, etc.) (range: 0-9)

cur_system_param_index

This is a more global version of the "current choices" for menus under "System". In some menus, this actually replaces the "current choice" variable. The range varies depending on which menu is active.

cur_num_system_params

This is global "System" mode count of the max value for the active menu.

selected_program_num

This is the "Selected" but not necessarily loaded program number.

running_program_num

This is the number of the currently running program.

operating_mode

This contains the current operating mode. The modes are defined as follows:

PROGRAM_MODE 0
EDIT_MODE 1
SYSTEM_MODE 2

cur_setup_num

This is the current "Setup" that is active. (range: 0-4)

setup

This is the current choice for the "System: Mode" (formerly "Setup") menu. (range: 0-9)

cur_bypass_patch_num

This is current choice of the bypass patch menu (Option under "System:Mode Bypass"). (range: 0-6)

cur_global_patch_num

This contains the current "Global Patch" number that is being edited. (range: 0-9)

cur_global_patch_menu

This contains the current choice for the "Global Patches" menu. (range: 0-2)

cur_ui_dump_type

This contains the currently selected MIDI dump type. (range: 0-21)

cur_cont_remap

This contains the currently selected "Controller Remap" choice (which controller's transmit value is being edited).

patch

This contains the current patch number being edited in the patch menu (range: 0-4)

cur_patch_num

This is the current parameter for the patching system. The range of this is 0-5 if a destination is defined otherwise the value is limited to 0-2.

cur_auto_store

This is the current setting for the auto store function. (0 == lose edits, 1 == prompt user for store before selecting and loading a new program).

cur_tempo

This contains the current choice for the "Edit:Tempo" menu. (range: 0-3)

cur_verify

This contains the current choice for the verify "Yes/No" menu. (range: 0-1)

cur_verify_store

This contains the current choice for the "Store: Yes/No" menu. (range: 0-1)

MPX G2 - Patches Dump

(Type 0x015F)

This dump allows you to get the patch data for all of the patches in the active program or send it to the system. There are 5 patches transmitted in this message which are each organized as follows:

Byte #	Description	# Bytes
1,2	Source Controller number	1
3,4	Source Minimum value	1
5,6	Source Middle value	1
7,8	Source Maximum Value	1
9,10	Destination Effect/Parameter type	1
11,12	Destination Parameter index	1
13-16	Destination Minimum value	2
17-20	Destination Middle value	2
21-24	Destination Maximum value	2

Refer to the Patch system data section of the "Program Dump" message for additional information about the fields of this dump.

MPX G2 - Soft Row Dump

(Type 0x0160)

This message allows you to get a dump of all the Soft value (soft row) data in the active program or send it to the system.

There are 10 Soft Row parameters which are stored in each program and are transferred with this packet, each containing the following:

Effect/Parameter type (1 byte)
Parameter index (1 byte)

Refer to the Soft Values section of the "Program Dump" message for detailed information about the contents of these fields.

MPX G2 - Display Units

This file contains descriptions of the display functions, which are used to display MPX G2 parameter values. The "Unit#" refers to the "Display Unit" ID which is part of the description of each parameter in the system. Typically, functions are organized in a function table that is indexed into using the Display Unit ID associated with a particular parameter.

Unit#	Name	Description
0x00	Mix Disp Units	Display in decimal followed by '%'
0x01	No Disp Units	Display in decimal
0x02	Waveform Disp Units	Use the "waveform_strings"
0x03	Percentage Disp Units	Display in decimal followed by '%'
0x04	On Off Disp Units	Use the "on_off_strings"
0x05	Log Lin Disp Units	Use the "log_lin_strings"
0x06	Note Disp Units	Use the "note_strings"
0x07	"Off" Decimal Disp Units	Display "Off" for 0, otherwise in decimal
0x08	Tempo Ratio Disp Units	Display high and low bytes as separate decimal numbers seperated by a ':'. Strip off the MSB of the high byte.
0x09	Hz Disp Units	Display in decimal followed by "Hz"
0x0a	Q Disp Units	Display in decimal 1/10ths units optionally followed by 'Q' (e.g. 10 would be "1.0 Q")
0x0b	Envelope Disp Units	Use the "envelope_mode_strings"
0x0c	Velocity Disp Units	Use the "velocity_mode_strings"
0x0d	Modulation Disp Units	Use the "modulation_mode_strings"
0x0e	Degree Disp Units	Display in decimal (optional degree symbol)
0x0f	Ms Or Time Sig Disp Units	Display high and low bytes as separate decimal numbers seperated by a ':'. Strip off the MSB of the high byte.
0x10	Load Mode Disp Units	Use the "load_mode_strings"
0x11	Lfo Mode Disp Units	Use the "lfo_mode_strings"
0x12	Adsr Mode Disp Units	Use the "adsr_mode_strings"
0x13	Rate Units Disp Units	If the third byte is 1, display "cycles:beat" otherwise display "Hz".
0x14	Time Units Disp Units	Use the "time_unit_strings" using the third byte as an index.
0x15	Fb Insert Disp Units	Use the "fb_insert_point_strings" using the option data as an index.
0x16	Tap Mode Disp Units	Use the "tap_mode_strings"
0x17	Rate Disp Units	Display in decimal 1/100ths units with optional "Hz" (e.g. 100 would be "1.00 Hz")
0x18	Midi Channel Disp Units	For values 0-15, display as decimal number plus 1. For value of 16, display "Off" For value of 17, display "Omni"
0x19	Midi Out Rate Disp Units	Use the "midi_out_rate_strings"
0x1a	Meter Disp Units	Use the "meter_strings"
0x1b	Sort Mode Disp Units	Use the "sort_mode_strings"
0x1c	Bypass Mode Disp Units	Use the "bypass_mode_strings"
0x1d	Mem Protect Disp Units	Use the "mem_protect_mode_strings"
0x1e	Patch Update Mode Disp units	Use the "patch_update_mode_strings"
0x1f	Sleep Mode Disp Units	Use the "sleep_mode_strings"
0x20	Mix Mode Disp Units	Use the "mix_mode_strings"
0x21	Program Load Disp Units	Use the "program_load_strings"
0x22	Clock Source Disp Units	Use the "clock_source_strings"
0x23	Audio Output Disp Units	Use the "audio_output_mode_strings"
0x24	Chan Stat Disp Units	Use the "chan_stat_mode_strings"
0x25	Alphanumeric Disp Units	Display the data as an ASCII string
0x26	Bpm Disp Units	Display in decimal followed by BPM
0x27	Tempo Source Disp Units	Use the "tempo_source_strings"
0x28	Cont Source Disp Units	Use the display strings in the Control Source table
0x29	Min Off Cont Source Disp Units	Same as previous except display "None" for the "min_value"
0x2a	Num Beats Disp Units	Display in decimal followed by "Beats"
0x2b	Options No Disp Units	Display in decimal using option data
0x2c	Ab Mode Disp Units	Use the "ab_mode_strings"
0x2d	Global Patch Dest Disp Units	Use the "global_patch_dest_strings"
0x2e	MIDI Dump Disp Units	Use the "midi_dump_strings"
0x2f	Eq Mode Disp Units	Use the "eq_mode_strings"
0x30	Name Disp Units	Display data as an ASCII string
0x31	Wah Type Disp Units	Use the "wah_type_strings" using the option data as an index
0x32	Input Mode Disp Units	Use the "input_mode_strings"
0x33	Arp Mode Disp Units	Use the "arpeggiator_mode_strings"
0x34	Env Source Disp Units	Use the "envelope_source_strings"
0x35	Size Disp Units	Divide the value by 2 and add 4. If LSB of value is 1, add ".5" else add ".0". Optionally add "M" or "Meters".
0x36	Treb Disp Units	Use "reverb_freq_strings"
0x37	Bassrt Disp Units	Display 'X' followed by "bass_rt_strings"
0x38	Crossover Disp Units	Use "reverb_freq_strings" offsetting the parameter value by 12. If value = 48 use "Flat".
0x39	Midrt Disp Units	If reverb algorithm is 1 or 5 use the "chamber_decay_strings", if 2 then use "hall_decay_strings", if 3 use "plate_decay_strings"
0x3a	Percent50 Disp Units	Multiply value by 2 and display in decimal followed by '%'.
0x3b	MSec Disp Units	Display in decimal (optionally follow with "ms")
0x3c	Spread Disp Units	Display in decimal. Note that if "Link" is turned on, the value is scaled
0x3d	Shape Disp Units	Display in decimal
0x3e	Slope Disp Units	If the value is < 16, a minus sign is displayed followed by 16 minus the value in decimal. Otherwise, a plus sign is displayed followed by the value minus 16 in decimal
0x3f	DPLevel Disp Units	If the value is 0, "Off" is displayed. If the value is the "max_value", "Full" is displayed. Otherwise, a minus sign is displayed, the value is multiplied by 2 and used as an offset into the "levelog_db" table, where the first display value is derived. The value is incremented then used again for an index into the table for the second character. Finally, the letters "dB" are tacked onto the end.
0x40	Duration Disp Units	140 is added to the value multiplied by 5 (140+(value*5)) and displayed as decimal with "ms" tacked onto the end.
0x41	Cromatics Note Disp Units	The value has 12 subtracted from it until is less than 11 with the octave incremented each time. The resulting "octave" number is displayed in hex followed by the note using what's left of the value as an index into "cromatic_note_strings".
0x42	AmbRtHc Disp Units	Use "ambience_RT_HC_strings"
0x43	Word No Disp Units	Display as a decimal word (2 bytes)
0x44	Dump Disp Units	Display "Dump"
0x45	Feet Disp Units	Display in decimal (optionally add "Feet")
0x46	Meters Disp Units	Display in decimal (optionally add "Meters")
0x47	Phase Disp Units	Use "phase_strings"
0x48	PDly Disp Units	Display in decimal (optionally add "ms")
0x49	Optimize Disp Units	Display in decimal using option data (optionally add "ms")
0x4a	Arp Velocity Source Disp Units	If value is < 128, display as a decimal number otherwise subtract 127 and use 0x28 Controller Source Display Units.
0x4b	Control Level Disp Units	Display the parameters name
0x4c	On/Off with bin Disp Units	Use "on_off_with_bin_strings"
0x4d	DSP Bypass Disp Units	Use "DSP_bypass_strings"
0x4e	Tone Disp Units	Use "tone_strings"
0x4f	MoJo Disp Units	Use "mojo_bypass_strings"
0x50	SX Bypass Disp Units	Use "SX_bypass_strings"
0x51	Feel Disp Units	Use "feel_strings"
0x52	Send Select Disp Units	Use "send_sel_strings"
0x53	Sum Mono Disp Units	Use "sum_mono_strings"
0x54	Left Insert Disp Units	Use "left_ins_strings"
0x55	Speaker Sim Bypass Disp Units	Use "spkr_sim_bypass_strings"
0x56	Right Insert CT Disp Units	Use "rt_ins_ct_strings"
0x57	Mix Insert Disp Units	Use "mix_insert_strings"
0x58	Mute Disp Units	Use "mute_strings"
0x59	Send Bypass Disp Units	Use "send_bypass_mode_strings"
0x5a	Post Bypass Disp Units	Use "post_bypass_mode_strings"
0x5b	Insert Mode Disp Units	Use "insert_mode_strings"
0x5c	Tuner Bypass Mode Disp Units	Use "tuner_bypass_mode_strings"
0x5d	Noise Gate Enable Disp Units	Use "noise_gate_enable_strings"
0x5e	Cabinet Disp Units	Use "cabinet_strings"
0x5f	Toe Patch Disp Units	Use "toe_patch_strings"
0x60	Key Disp Units	Use "cromatic_note_strings"
0x61	Scale Disp Units	Use "scale_strings"
0x62	Interval Disp Units	Use "interval_strings"
0x63	Pitch Detect Source	Use "pitch_detect_source_strings"
0x64	Insert Type	Use "insert_type_strings"
0x65	Speaker Frequency	Use "speaker_freq_strings"
0x66	Tuner Calibration	Display the string "A=" followed by the string in the "tuner_cal_strings" identified by the parameter.
0x67	Patch Destination	See "Patch Destination Display Units" below.
0x68	Controller Group and Source	Use the display strings in the Control Source table
0x69	Soft Row Assign	See Patch Destination Display Units" below.
0x6a	Diatonic Input	Use the "noise_gate_enable_strings"
0x6b	Program Name	Each byte of the parameter represents an ASCII character.
0x80	Level Disp Units	If value = "min_value", display "Off" otherwise display a signed decimal number followed by "dB".
0x81	Pitch Disp Units	Display as signed decimal number in 100ths (100 = "1.00")
0x82	Bipolar Percentage Disp Units	Display as signed decimal number followed by "%".
0x83	Bipolar No Disp Units	Display as signed decimal number
0x84	Pan Disp Units	If value = 0, display 'C' or "Center". If value is negative (MSB = 1), display (FF - value) + 1 in decimal followed by 'L' or "Left". If the value is positive, display the value in decimal followed by 'R' or "Right".
0x85	Bipolar Word Disp Units	Display in decimal word (2 bytes)
0x86	FF "None" No Disp Units	If the value is FF, display "None" otherwise, display "CC" followed by the value in decimal.
0x87	Bipolar Degrees Disp Units	Display in signed decimal (optionally add the degree symbol or the word "degrees")
0x88	Bal Disp Units	Display in signed decimal.
0x89	Tone Level Disp Units	Display in signed decimal followed by "dB".
0x8a	Signed Bargraph	Display a signed number
0x8b	Tuner Offset	Display the string "Cents=" followed by a signed version of the number.
0x8c	Inverted Level	Same as "Level Disp Units"

Patch Destination Display Units

This display unit provides a list of target parameters that can be controlled by a patch. Because the parameter list that can be controlled by a patch is dependent on which algorithms are currently loaded, the current value of the destination parameter must be used to count patchable parameters on the tree. For example if the destination parameter had a value of 5, FX 1 had no effect loaded, FX 2 had 3 patchable parameters and Chorus had 8 patchable parameters, the third Chorus parameter would be selected as the destination.

MPX G2 Control Sources

Master List of Controllers

The following table lists all internal (Ctls) and MIDI controllers:

Number	Display	Description
0x00	Unassigned	Null controller
0x01	Off	always 0
0x02	On	always 1

0x03	Knob	adjust knob
0x04	Puls1	lfo1 pulse/square wave output
0x05	Tri1	lfo1 triangle wave output
0x06	Sine1	lfo1 sinewave output
0x07	Cos1	lfo1 cosine wave output
0x08	Puls2	lfo2 pulse/squarewave output
0x09	Tri2	lfo2 triangle wave output
0x0A	Sine2	lfo2 sinewave output
0x0B	Cos2	lfo2 cosine wave output
0x0C	Rand	random number generator
0x0D	Env	envelope follower
0x0E	InLvl	signal level at main input
0x0F	ALvl	signal level at aux inputs
0x10	A/B	AB Controller (A=0, B=127)
0x11	ATrg	pulse whenever A/B changes from B to A
0x12	BTrg	pulse whenever A/B changes from A to B
0x13	ABTrg	pulse whenever A/B changes
0x14	Pedal	foot pedal (rear panel jack)
0x15	Tog1	latched output of foot switch 1
0x16	Tog2	latched output of foot switch 2
0x17	Tog3	latched output of foot switch 3
0x18	Sw1	momentary output of foot switch 1
0x19	Sw2	momentary output of foot switch 2
0x1A	Sw3	momentary output of foot switch 3
0x1C	CC1	MIDI controller 1
0x1D	CC2	MIDI controller 2
...		(sequential except 32 which is skipped)
0x90	CC119	MIDI controller 119
0x91	Bend	pitch bend
0x92	Touch	after touch
0x93	Vel	note on velocity
0x94	Last•	last note (“•”is place holder for a note special character)
0x95	Low•	low note
0x96	High•	high note
0x97	Tempo	MIDI/internal tempo (40-400BPM is converted to source range of 0=127)
0x98	Cmnds	Start, Stop, Continue converted to switch; start/continue = 127, stop = 0
0x99	Gate	on as long as at least one MIDI note is on
0x9A	Trig	pulse whenever a new MIDI note on is detected
0x9B	LGate	on only when more than one MIDI note is on
0x9C	TSw	toggled on and off by after touch
0x9D	Toe	the toe switch under the pedal on the R1 remote

Global Patch controllers

Uses the numbers from the list above except the items before "Pedal" are excluded.

0x14	Pedal	foot pedal
...		
0x9D	Toe	the toe switch under the pedal on the R1 remote

Bypass controllers

Uses the numbers from the list above except the items before "Tog 1" are excluded.

0x15	Tog1	latched output of foot switch 1
...		
0x9D	Toe	the toe switch under the pedal on the R1 remote